



Faculté des Sciences Ben M'Sick (FSBM)

Université Hassan II de Casablanca

Rapport de Projet de Fin d'Études

Système de gestion des soutenances et des jurys

Réalisé par :

Zaitouni Nourelislam

Taoudi Ebourki Abdelaziz

Encadré par :

Pr. Mohammed AIT DAOUD

Co-encadré par :

Dr. Salma SAMMAH

Licence Fondamentale – MIP

Année Universitaire 2025–2026

À nos parents, pour leur soutien indéfectible.

À nos professeurs, pour leur encadrement et leur savoir.

À tous ceux qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre encadrant, **Pr. Mohammed AIT DAOUD**, pour sa disponibilité, ses conseils avisés et son accompagnement tout au long de ce projet.

Nous remercions également notre co-encadrante, **Dr. Salma SAMMAH**, pour son soutien technique et ses précieuses orientations.

Nos remerciements s'adressent aussi à l'ensemble du corps professoral du département MIP de la Faculté des Sciences Ben M'Sick pour la qualité de la formation dispensée durant ces trois années de licence.

Enfin, nous remercions nos familles et nos proches pour leur patience et leurs encouragements constants.

Résumé

L'organisation des soutenances en milieu universitaire constitue un processus complexe mobilisant de multiples acteurs – étudiants, encadrants, membres de jury et personnel administratif – autour de contraintes temporelles, spatiales et réglementaires. Le présent projet vise à concevoir et développer une application web complète pour la gestion centralisée, automatisée et fiable des soutenances et des jurys au sein de la Faculté des Sciences Ben M'Sick. Le système repose sur une architecture moderne à trois couches : un frontend React, un backend Spring Boot et une base de données MySQL. Il couvre l'ensemble du cycle de vie d'une soutenance, de la planification des créneaux et de l'affectation des jurys jusqu'à la saisie des évaluations et la génération automatique des documents administratifs (convocations, procès-verbaux, fiches d'évaluation). Une attention particulière est portée à la détection des conflits horaires et à l'équilibrage de la charge des enseignants. Ce rapport présente l'étude de l'existant, l'analyse des besoins, la conception UML, l'architecture technique et la modélisation de la base de données du système.

Mots-clés : Gestion de soutenances, planification, jury, évaluation, Spring Boot, React, JWT, UML, système d'information universitaire.

Abstract

Organizing thesis defenses in a university environment is a complex process involving multiple actors – students, supervisors, jury members, and administrative staff – around temporal, spatial and regulatory constraints. This project aims to design and develop a complete web application for the centralized, automated and reliable management of defenses and juries at the Faculty of Sciences Ben M'Sick. The system is based on a modern three-tier architecture : a React frontend, a Spring Boot backend, and a MySQL database. It covers the entire lifecycle of a defense, from slot planning and jury assignment to evaluation entry and automatic generation of administrative documents (summons, minutes, evaluation sheets). Special attention is given to conflict detection and teacher workload balancing. This report presents the study of the current system, requirements analysis, UML design, technical architecture, and database modeling.

Keywords : Defense management, planning, jury, evaluation, Spring Boot, React, JWT, UML, university information system.

Table des matières

Remerciements	2
Résumé	3
Abstract	3
Liste des figures	7
Liste des tableaux	8
Liste des acronymes	10
Introduction Générale	11
Contexte général	11
Problématique	11
Objectifs du projet	11
Méthodologie suivie	12
Structure du rapport	12
1 Présentation de l'organisme d'accueil	13
1.1 Introduction	13
1.2 Présentation générale de l'organisme	13
1.2.1 Historique et fiche signalétique	13
1.2.2 Missions et activités	13
1.2.3 Secteurs d'intervention	13
1.3 Structure organisationnelle	14
1.3.1 Organigramme	14
1.3.2 Description des départements	14
1.4 Environnement du stage	14
1.4.1 Département d'accueil	14
1.4.2 Contexte du projet au sein de l'organisme	15
1.5 Conclusion	15
2 Étude de l'existant et technologies utilisées	16
2.1 Introduction	16
2.2 Analyse de l'existant	16
2.2.1 Description du système actuel	16
2.2.2 Processus actuel détaillé	16
2.2.3 Analyse critique	19
2.3 Étude des solutions similaires	19
2.3.1 Systèmes universitaires généraux (ERP)	19
2.3.2 Outils de planification générique	19
2.3.3 Solutions open source (GitHub)	20
2.4 Choix technologiques	20
2.4.1 Frontend	20
2.4.2 Backend	20
2.4.3 Base de données	21

2.4.4	Outils de développement	21
2.5	Architecture générale retenue	21
2.6	Conclusion	21
3	Analyse des besoins et conception du système	23
3.1	Introduction	23
3.2	Présentation générale du projet	23
3.3	Cahier des charges	23
3.3.1	Acteurs du système	23
3.3.2	Besoins fonctionnels	23
3.3.3	Besoins non fonctionnels	24
3.4	Règles de gestion	25
3.5	Modules du système	25
3.5.1	Module 1 : Authentification et gestion des rôles	25
3.5.2	Module 2 : Gestion académique	25
3.5.3	Module 3 : Planification	25
3.5.4	Module 4 : Gestion des jurys	25
3.5.5	Module 5 : Évaluation	25
3.5.6	Module 6 : Génération documentaire	26
3.5.7	Module 7 : Tableau de bord	26
3.6	Conception UML	26
3.6.1	Diagrammes de cas d'utilisation	26
3.6.2	Diagramme de classes	32
3.6.3	Cycle de vie d'une soutenance	34
3.6.4	Diagrammes d'activité	35
3.6.5	Diagrammes de séquence	37
3.7	Architecture technique	44
3.7.1	Architecture en couches	44
3.7.2	Architecture technique des composants	45
3.7.3	Sécurité et Authentification JWT	45
3.8	Modélisation de la base de données	46
3.8.1	Dictionnaire des données	46
3.9	Conception de l'interface utilisateur	51
3.9.1	Plan de navigation	51
3.9.2	Prototypes d'interface	51
3.9.3	Modèle de convocation officielle	56
3.10	Conclusion	58
4	Réalisation et développement	59
4.1	Introduction	59
4.2	Environnement de développement	59
4.3	Implémentation du backend (Spring Boot)	59
4.4	Implémentation du frontend (React)	59
4.4.1	Architecture détaillée du backend	59
4.4.2	Architecture détaillée du frontend	61
4.5	Implémentation des modules fonctionnels	61
4.5.1	Module 1 : Authentification et gestion des rôles	62
4.5.2	Module 2 : Gestion académique	63

4.5.3	Module 3 : Planification	63
4.5.4	Module 4 : Gestion des jurys	65
4.5.5	Module 5 : Évaluation	66
4.5.6	Module 6 : Génération documentaire	67
4.5.7	Module 7 : Tableau de bord	67
4.6	Extraits de code représentatifs	67
4.6.1	1. Génération JWT (Backend)	67
4.6.2	2. Vérification de conflit : Double réservation enseignant	68
4.6.3	3. React Query : Hook personnalisé (Frontend)	69
4.7	Interfaces utilisateur	70
4.7.1	Écran de connexion	71
4.7.2	Defense Designer (Coordinateur)	71
4.7.3	Tableau de bord Administrateur	72
4.7.4	Formulaire d'évaluation (Enseignant)	72
4.7.5	Espace Étudiant	73
4.7.6	Impression : Convocation	73
4.8	Difficultés rencontrées et solutions	74
4.9	Conclusion	74
5	Tests et validation	75
5.1	Introduction	75
5.2	Stratégie de test	75
5.3	Scénarios de test	75
5.4	Validation des fonctionnalités	76
5.5	Évaluation globale du système	76
5.6	Limites du projet	76
5.7	Conclusion	77
	Conclusion Générale et Perspectives	78
	Bilan du travail réalisé	78
	Apports personnels et compétences acquises	78
	Perspectives d'évolution	78
	Bibliographie	79

Table des figures

1	Processus actuel – Phase 1 : Planification	17
2	Processus actuel – Phase 2 : Exécution	18
3	Architecture trois couches du système	21
4	Hiérarchie générale des acteurs	27
5	Cas d'utilisation : Administration du système	28
6	Cas d'utilisation : Coordination des soutenances	29
7	Cas d'utilisation : Enseignant et Jury	31
8	Cas d'utilisation : Espace Étudiant	32
9	Diagramme de classes global du système	33
10	Diagramme d'états : Cycle de vie d'une soutenance	35
11	Diagramme d'activité : Planification de session	36
12	Diagramme d'activité : Processus d'évaluation	37
13	Diagramme de séquence : Authentification	38
14	Diagramme de séquence : Planification de soutenance	39
15	Diagramme de séquence : Affectation du jury	40
16	Diagramme de séquence : Saisie d'évaluation	41
17	Diagramme de séquence : Génération PDF	41
18	Diagramme de séquence : Import massif de données	42
19	Diagramme de séquence : Validation finale et notifications	43
20	Diagramme de séquence : Réinitialisation de mot de passe	44
21	Architecture générale en trois couches	44
22	Architecture technique des composants	45
23	Modèle Logique de Données (MLD)	46
24	Sitemap et plan de navigation	51
25	Interface de connexion	52
26	Interface : Tableau de bord Administrateur	53
27	Interface : Tableau de bord Coordinateur	54
28	Interface : Tableau de bord Enseignant	55
29	Interface : Tableau de bord Étudiant	56
30	Spécimen de convocation officielle (Format A4)	57
31	Architecture en paquets du backend Spring Boot	60
32	Architecture du frontend React	61
33	Diagramme de séquence : Processus d'authentification JWT	62
34	Diagramme de séquence : Processus de planification	64
35	Diagramme de séquence : Affectation d'un jury	65
36	Diagramme de séquence : Saisie d'une évaluation	66
37	Interface : Page de connexion	71
38	Interface : Defense Designer (Planification)	72
39	Interface : Tableau de bord Admin	72
40	Interface : Formulaire d'évaluation	73
41	Interface : Espace étudiant	73
42	Interface : Modèle de convocation (impression)	74

Liste des tableaux

1	Fiche signalétique de la FSBM	13
2	Analyse des systèmes ERP académiques	19
3	Analyse des outils de planification génériques	19
4	Analyse de projets open source similaires	20
5	Description des cas d'utilisation – Administrateur	28
6	Description des cas d'utilisation – Coordinateur	30
7	Description des cas d'utilisation – Enseignant	31
8	Description des cas d'utilisation – Étudiant	32
9	Dictionnaire – Table utilisateurs	47
10	Dictionnaire – Table étudiants	47
11	Dictionnaire – Table enseignants	47
12	Dictionnaire – Table administrateurs / coordinateurs	47
13	Dictionnaire – Table sessions_globales	48
14	Dictionnaire – Table projets	48
15	Dictionnaire – Table salles	48
16	Dictionnaire – Table soutenances	48
17	Dictionnaire – Table membres_jury	49
18	Dictionnaire – Table evaluations	49
19	Dictionnaire – Table resultats_soutenance	49
20	Dictionnaire – Table convocations	50
21	Dictionnaire – Table indisponibilites	50
22	Dictionnaire – Table notifications	50
23	Dictionnaire – Table reset_tokens	50
24	Types de conflits détectés	66
25	Architecture des pages par rôle	71
26	Infrastructure de test	75

Liste des acronymes

Acronyme	Définition
ADMIN	Administrateur
API	Application Programming Interface – Interface de programmation applicative
BDD	Base de Données
CDC	Cahier des Charges
CI/CD	Continuous Integration / Continuous Deployment – Intégration continue / Déploiement continu
CRUD	Create, Read, Update, Delete – Opérations de base sur les données
CSV	Comma-Separated Values – Valeurs séparées par virgule (format de fichier)
CSS	Cascading Style Sheets – Feuilles de style en cascade
E2E	End-to-End – De bout en bout (tests)
FSBM	Faculté des Sciences Ben M'Sick
HTML	HyperText Markup Language – Langage de balisage hypertexte
HTTP	HyperText Transfer Protocol – Protocole de transfert hypertexte
HTTPS	HyperText Transfer Protocol Secure – HTTP sécurisé
IDE	Integrated Development Environment – Environnement de développement intégré
JPA	Java Persistence API – API de persistance Java
JWT	JSON Web Token – Jeton web JSON (authentification)
JS	JavaScript – Langage de programmation web
JSON	JavaScript Object Notation – Format d'échange de données
MIP	Mathématiques, Informatique et Physique – Filière de licence
MVC	Model-View-Controller – Modèle-Vue-Contrôleur (architecture logicielle)
MySQL	Système de gestion de base de données relationnelle
PDF	Portable Document Format – Format de document portable
PFE	Projet de Fin d'Études
REST	Representational State Transfer – Style d'architecture pour APIs web

Introduction Générale

Contexte général

L'organisation des soutenances en milieu universitaire est une activité critique qui se heurte aujourd'hui à plusieurs défis. La multiplicité des acteurs – étudiants, encadrants, membres de jury et services administratifs – nécessite une coordination fine et rigoureuse. Pourtant, les processus traditionnels reposent encore largement sur des tableurs Excel, des échanges d'e-mails massifs et des documents papier, engendrant une perte d'efficacité significative et des erreurs de planification fréquentes.

Dans ce contexte, la faculté des Sciences Ben M'Sick (FSBM) de l'Université Hassan II de Casablanca, à travers son département de Mathématiques, Informatique et Physique (MIP), cherche à moderniser ses outils de gestion académique. L'objectif est de centraliser les données et d'automatiser les flux de travail via une plateforme web dédiée, capable de répondre aux exigences croissantes de qualité et de réactivité.

Problématique

La coordination des soutenances dans un département universitaire fait face à une complexité croissante. Les contraintes à respecter sont multiples et souvent interdépendantes. La disponibilité des enseignants, qui interviennent à la fois comme encadrants et comme membres de jury, doit être prise en compte pour éviter les doubles affectations horaires. La réservation des salles, dont les capacités varient, nécessite une allocation cohérente avec le nombre de participants attendus.

Par ailleurs, la production des documents administratifs accompagnant chaque soutenance – convocations, fiches d'évaluation – représente une charge de travail récurrente et chronophage. La saisie et le suivi des évaluations, la transmission des notes aux services compétents et la génération des rapports de synthèse complètent ce tableau.

Le cœur de la problématique réside donc dans l'absence d'un outil unifié capable de gérer l'ensemble de ces aspects de manière intégrée et automatisée. L'organisation manuelle actuelle, basée sur des outils génériques non adaptés à cette finalité spécifique, génère des inefficacités, des erreurs et une traçabilité insuffisante.

Objectifs du projet

Objectif principal: Développer une application web complète et fonctionnelle permettant la gestion centralisée, automatisée et fiable de l'organisation des soutenances et des jurys au sein d'un établissement universitaire.

Plusieurs objectifs spécifiques déclinent cette vision générale :

- **Gestion des acteurs académiques :** étudiants, encadrants, membres de jury et responsables pédagogiques.
- **Planification :** gestion des dates, horaires et salles, avec affectation des jurys et détection automatique des conflits.

- **Production documentaire** : génération automatique des convocations, fiches d'évaluation et procès-verbaux.
- **Évaluation** : saisie des notes et observations, enregistrement des délibérations et production d'états récapitulatifs.

Méthodologie suivie

La réalisation de ce projet a suivi une démarche structurée en plusieurs phases :

1. **Analyse de l'existant** : étude des processus actuels d'organisation des soutenances, identification des lacunes et définition des besoins.
2. **Conception** : modélisation UML (cas d'utilisation, diagramme de classes, diagrammes de séquence et d'activité), conception de l'architecture technique et modélisation de la base de données.
3. **Réalisation** : implémentation du backend Spring Boot et du frontend React, développement des modules fonctionnels.
4. **Tests et validation** : tests fonctionnels, tests d'intégration et validation utilisateur.

Structure du rapport

Le présent rapport s'articule autour de cinq chapitres :

- **Chapitre 1** présente l'organisme d'accueil, la Faculté des Sciences Ben M'Sick, son organisation et le contexte du stage.
- **Chapitre 2** dresse un état des lieux du système actuel, analyse les solutions existantes et justifie les choix technologiques retenus.
- **Chapitre 3** détaille l'analyse des besoins, la modélisation UML et la conception complète du système.
- **Chapitre 4** décrit la réalisation, l'implémentation technique et les difficultés rencontrées.
- **Chapitre 5** présente la stratégie de test, les scénarios de validation et les limites du projet.

Le rapport se termine par une conclusion générale dressant le bilan du travail accompli et les perspectives d'évolution.

1 Présentation de l'organisme d'accueil

1.1 Introduction

Ce chapitre présente l'environnement dans lequel s'est déroulé notre projet de fin d'études. Il décrit la Faculté des Sciences Ben M'Sick (FSBM) – son histoire, ses missions, sa structure organisationnelle – ainsi que le département d'accueil et le cadre spécifique du stage. Cette contextualisation permet de mieux comprendre les enjeux institutionnels et pédagogiques qui ont motivé la réalisation de notre système de gestion des soutenances.

1.2 Présentation générale de l'organisme

1.2.1 Historique et fiche signalétique

La Faculté des Sciences Ben M'Sick (FSBM) est l'une des composantes de l'Université Hassan II de Casablanca, l'une des plus grandes universités du Maroc. Située dans l'arrondissement de Ben M'Sick, à Casablanca, elle a été créée pour répondre à la demande croissante de formation scientifique dans la région du Grand Casablanca.

Information	Détail
Nom officiel	Faculté des Sciences Ben M'Sick (FSBM)
Université de rattachement	Université Hassan II de Casablanca
Adresse	Avenue Commandant Driss El Harti, Ben M'Sick, Casablanca
Année de création	1997
Type	Établissement public d'enseignement supérieur
Domaines	Sciences exactes, Sciences et technologies

TABLE 1 – Fiche signalétique de la FSBM

1.2.2 Missions et activités

La FSBM a pour missions principales :

- **La formation initiale** : dispenser un enseignement supérieur de qualité dans les disciplines scientifiques fondamentales et appliquées aux niveaux Licence, Master et Doctorat.
- **La recherche scientifique** : mener des activités de recherche au sein de ses laboratoires et équipes de recherche dans divers domaines des sciences exactes.
- **L'insertion professionnelle** : préparer les étudiants à leur insertion dans le monde professionnel à travers des formations adaptées aux besoins du marché du travail.
- **Le rayonnement académique** : contribuer au développement scientifique et technologique de la région et du pays.

1.2.3 Secteurs d'intervention

La faculté couvre un large spectre de disciplines scientifiques :

- **Mathématiques** : algèbre, analyse, probabilités, statistiques, mathématiques appliquées.
- **Informatique** : développement logiciel, bases de données, réseaux, intelligence artificielle.

- **Physique** : physique fondamentale, électronique, mécanique, énergétique.
- **Chimie** : chimie organique, inorganique, chimie analytique, biochimie.
- **Sciences de la vie** : biologie moléculaire, biotechnologie, écologie.
- **Sciences de la Terre** : géologie, géophysique, géomatique.

1.3 Structure organisationnelle

1.3.1 Organigramme

La FSBM est administrée par un Doyen, assisté par des Vice-Doyens et un Secrétaire Général. La structure s'articule autour de plusieurs organes et entités :

- **Conseil de faculté** : instance délibérante composée d'enseignants-chercheurs, d'étudiants et de représentants administratifs.
- **Commissions pédagogiques** : chargées de l'organisation et du suivi des filières de formation.
- **Départements pédagogiques** : entités regroupant les enseignants-chercheurs par discipline.
- **Laboratoires de recherche** : structures dédiées à la recherche scientifique.
- **Services administratifs** : scolarité, ressources humaines, finances, bibliothèque.

1.3.2 Description des départements

La faculté est structurée en plusieurs départements pédagogiques, chacun responsable d'une ou plusieurs filières de formation :

- **Département de Mathématiques** : propose les licences en Mathématiques Fondamentales et Mathématiques Appliquées, ainsi que des masters en mathématiques.
- **Département d'Informatique** : responsable de la filière Licence MIP (Mathématiques, Informatique et Physique) et de plusieurs masters en informatique.
- **Département de Physique** : offre des licences en Physique Fondamentale et Physique des Matériaux.
- **Département de Chimie** : gère les licences en Chimie Fondamentale et Chimie Analytique.
- **Département de Biologie** : propose des licences en Biologie Moléculaire et Biotechnologie.
- **Département de Géologie** : offre des formations en Sciences de la Terre.

Chaque département est dirigé par un chef de département élu, assisté d'un conseil de département. Il coordonne les activités pédagogiques, les emplois du temps et la répartition des enseignements.

1.4 Environnement du stage

1.4.1 Département d'accueil

Notre projet de fin d'études s'est déroulé au sein de la filière **Licence Fondamentale MIP (Mathématiques, Informatique et Physique)** du Département d'Informatique. Cette filière vise à

former des étudiants polyvalents possédant une solide base en mathématiques et une maîtrise des technologies de l'information. La formation couvre des domaines variés tels que le développement logiciel, les bases de données, les réseaux, l'algorithmique avancée et l'intelligence artificielle.

Le corps enseignant de la filière est composé de professeurs de l'enseignement supérieur, de professeurs habilités et de professeurs assistants, couvrant l'ensemble des spécialités du programme.

1.4.2 Contexte du projet au sein de l'organisme

Le projet de système de gestion des soutenances et des jurys s'inscrit dans une volonté de modernisation des processus administratifs et pédagogiques au sein de la faculté. Actuellement, l'organisation des soutenances – qu'il s'agisse des soutenances de PFE, de mémoires de master ou de projets de licence – repose sur des méthodes manuelles et des outils bureautiques génériques.

Le département MIP, confronté à un volume croissant d'étudiants et à une complexité accrue dans la coordination des jurys, a identifié le besoin d'un outil informatique dédié. Ce projet a donc été proposé par **Pr. Mohammed AIT DAOUD** dans le cadre des projets de fin d'études de la licence MIP, avec pour objectif de fournir une solution concrète et opérationnelle répondant aux besoins réels du département.

1.5 Conclusion

La Faculté des Sciences Ben M'Sick constitue un cadre académique riche et diversifié, offrant des formations scientifiques de qualité dans un large éventail de disciplines. Le département MIP, par sa proximité avec les technologies de l'information, est naturellement le lieu idéal pour le développement d'une solution numérique de gestion des soutenances. Ce contexte a directement inspiré les choix technologiques et architecturaux qui seront présentés dans les chapitres suivants.

2 Étude de l'existant et technologies utilisées

2.1 Introduction

Ce chapitre présente une analyse approfondie du système actuel d'organisation des soutenances au sein de la faculté, identifie ses lacunes et compare les solutions existantes sur le marché. Il détaille ensuite les choix technologiques retenus pour le développement de notre application en justifiant chaque décision. Enfin, l'architecture générale du système est présentée.

2.2 Analyse de l'existant

2.2.1 Description du système actuel

L'organisation actuelle des soutenances repose sur une combinaison d'outils bureautiques génériques et de processus manuels :

- **Outils** : utilisation intensive de tableurs **Excel** pour le suivi des étudiants et des plannings, et de **documents papier** pour les évaluations.
- **Communication** : échanges fragmentés par **courrier électronique** pour la collecte des disponibilités et l'envoi des convocations.
- **Logistique** : gestion indépendante des salles via des services tiers, souvent sans visibilité directe sur les occupations.
- **Évaluation** : saisie manuelle des notes sur fiches imprimées, suivie d'une re-saisie numérique pour le calcul des moyennes.

2.2.2 Processus actuel détaillé

L'organisation d'une session de soutenances suit un cheminement complexe divisé en deux grandes phases opérationnelles.

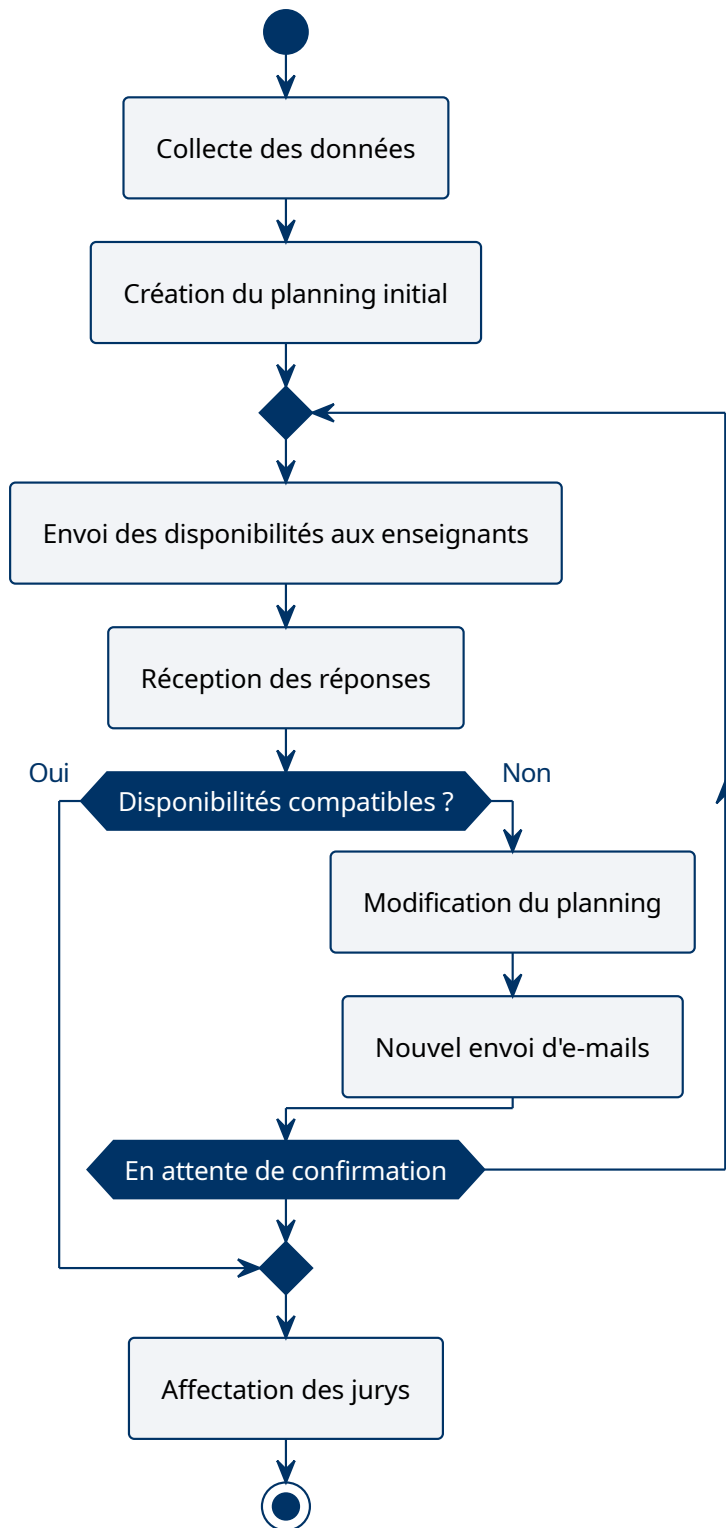


FIGURE 1 – Processus actuel – Phase 1 : Planification

La Figure 1 illustre la nature intrinsèquement itérative de la phase de planification. Elle repose sur une communication asynchrone par e-mail qui génère fréquemment des goulots d'étranglement : la centralisation manuelle des informations (étudiants, sujets, encadrants) et la négociation des créneaux imposent de multiples allers-retours avant d'aboutir à un planning acceptable.

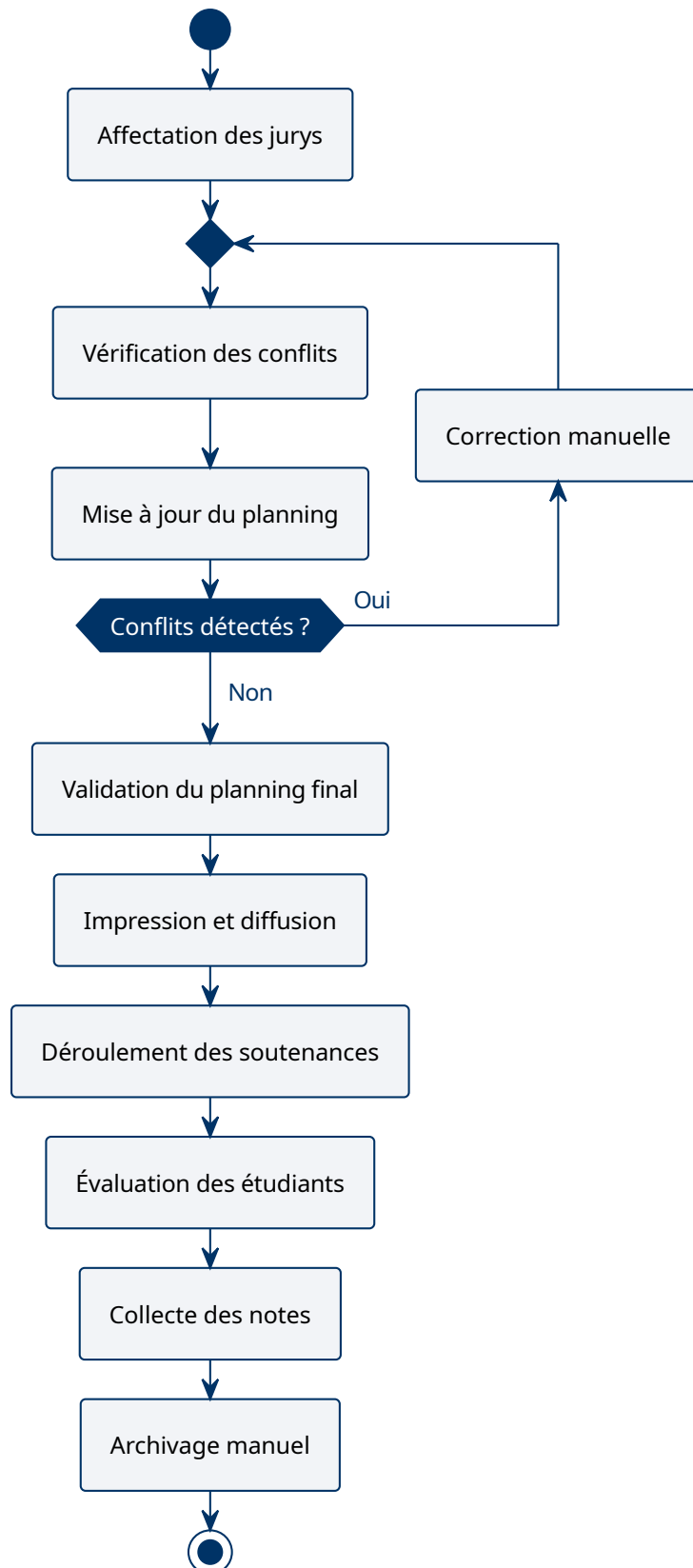


FIGURE 2 – Processus actuel – Phase 2 : Exécution

La Figure 2 détaille la transition vers la phase d'exécution. Cette étape est critique car elle comporte une vérification manuelle des conflits (double affectation d'un enseignant ou d'une

salle) avant la diffusion officielle. Le cycle se termine par une collecte physique des notes et un archivage souffrant d'un manque de centralisation numérique.

2.2.3 Analyse critique

Avantages. Le système actuel présente certains avantages dans des contextes de très petite taille : flexibilité d'adaptation à des cas particuliers, absence de dépendance technologique et charge de travail acceptable pour un volume très limité de soutenances.

Inconvénients. Les inconvénients deviennent prépondérants dès que le volume augmente. Les erreurs de planification sont fréquentes et souvent détectées tardivement, provoquant des annulations de dernière minute. La traçabilité des décisions est faible, la charge du coordinateur est considérable en fin de semestre, et la ressaisie répétée des données multiplie les risques d'erreurs de transcription.

2.3 Étude des solutions similaires

Plusieurs solutions logicielles existent sur le marché ou sous forme de projets open source. Les Tableaux 2, 3 et 4 présentent une analyse comparative qui a guidé le positionnement de notre projet.

2.3.1 Systèmes universitaires généraux (ERP)

Solution	Description	Adéquation
Apogée (Amue)	Système de gestion des étudiants utilisé dans les universités françaises	Trop généraliste ; gestion des soutenances non native
Aurion	ERP académique couvrant scolarité, stages et PFE	Module soutenance limité, coût élevé, complexité d'intégration
Koha / Aleph	Systèmes bibliothèques universitaires	Hors sujet

TABLE 2 – Analyse des systèmes ERP académiques

2.3.2 Outils de planification générique

Solution	Description	Limites pour ce besoin
Doodle	Sondages de disponibilité	Pas de gestion des jurys ni de génération PDF
Google Calendar	Calendrier partagé	Pas de logique métier académique, pas de convocations
Microsoft Teams	Gestion de réunions	Non adapté à la logique jury/soutenance

TABLE 3 – Analyse des outils de planification génériques

2.3.3 Solutions open source (GitHub)

Projet	Lien GitHub	Limites identifiées
appliSoutenance	cdegrel/appliSoutenance	Focalisé sur la notation en séance ; ne gère pas la planification amont ni les salles
PFE-Management	khaireddine24/PFE-MS	Gestion administrative de base ; absence de détection de conflits et de génération PDF
Jury (HackUTD)	hackutd/jury	Adapté aux hackathons ; incompatible avec les rôles académiques (Président, Rapporteur)

TABLE 4 – Analyse de projets open source similaires

Conclusion de l'analyse : Comme le montrent les Tableaux 2, 3 et 4, aucune solution existante ne réunit simultanément les quatre points suivants :

- Gestion complète du cycle de soutenance (planification → évaluation).
- Composition et affectation de jurys avec détection de conflits.
- Génération automatique de convocations et fiches d'évaluation.
- Adaptation au contexte marocain (filières, CNE, structure universitaire locale).

2.4 Choix technologiques

Le choix des technologies est une étape cruciale qui conditionne la réussite du projet. Chaque décision a été prise en fonction des besoins spécifiques du système, des compétences de l'équipe et des contraintes du projet.

2.4.1 Frontend

React.js a été retenu pour le développement de l'interface utilisateur. Ce choix se justifie par :

- **Réactivité** : son modèle de composants et le DOM virtuel offrent une expérience utilisateur fluide et sans rechargement de page.
- **Écosystème riche** : une vaste communauté, des bibliothèques matures et des outils de développement performants.
- **Compatibilité REST** : s'intègre parfaitement avec une API REST Spring Boot.

L'interface est conçue avec **Tailwind CSS** et **Shadcn UI** pour un design responsive, moderne et cohérent.

2.4.2 Backend

Spring Boot (Java) a été choisi pour la couche serveur pour les raisons suivantes :

- **Robustesse** : framework éprouvé pour les applications d'entreprise, offrant une sécurité et une stabilité élevées.

- **Spring Security** : intégration native pour la gestion de l'authentification JWT et le contrôle d'accès par rôles.
- **Spring Data JPA** : facilite l'accès aux données et la gestion des entités relationnelles.
- **Architecture MVC** : séparation claire des préoccupations (Contrôleurs, Services, Repositories).
- **Documentation et communauté** : large base de connaissances et support actif.

2.4.3 Base de données

MySQL a été sélectionné comme système de gestion de base de données :

- **Relationnel** : parfaitement adapté à la structure de données fortement interconnectée du système (étudiants, projets, jurys, soutenances, évaluations).
- **Performance** : fiable et performant pour les charges de travail académiques.
- **Intégration** : compatibilité native avec Spring Data JPA et Hibernate.
- **Coût** : solution open source mature sans frais de licence.

2.4.4 Outils de développement

- **Git** : versionnement du code source et collaboration via GitHub.
- **VS Code** : environnement de développement intégré pour le frontend.
- **IntelliJ IDEA** : IDE dédié au développement Spring Boot.
- **Postman** : test et documentation des API REST.
- **Maven** : gestion des dépendances et construction du projet backend.

2.5 Architecture générale retenue

L'application adopte une architecture web en trois couches classique (Figure 3), favorisant la modularité et la séparation des préoccupations :

- **Couche Frontend** : interface utilisateur React, affichage des données et capture des interactions.
- **Couche Backend** : logique métier Spring Boot, règles de gestion, détection des conflits et génération des documents.
- **Couche Base de données** : persistance MySQL, intégrité et sécurité des informations du système.

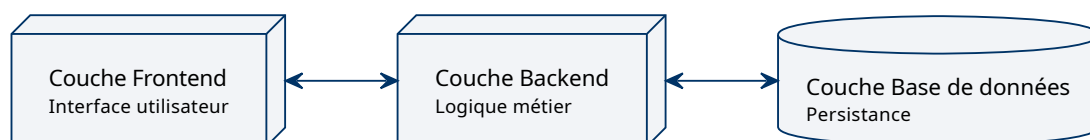


FIGURE 3 – Architecture trois couches du système

2.6 Conclusion

Ce chapitre a dressé un état des lieux complet du système actuel d'organisation des soutenances, mettant en évidence ses limites opérationnelles. L'analyse des solutions existantes a confirmé

l'absence d'un outil répondant spécifiquement aux besoins de notre contexte. Les choix technologiques – React, Spring Boot, MySQL – ont été justifiés et l'architecture à trois couches a été présentée comme fondation du système. Le chapitre suivant détaillera l'analyse des besoins fonctionnels et la conception complète du système.

3 Analyse des besoins et conception du système

3.1 Introduction

Ce chapitre constitue le cœur de la phase d'analyse et de conception du système. Il définit le cahier des charges fonctionnel, présente les règles de gestion qui encadrent le système, détaille la modélisation UML complète – des cas d'utilisation aux diagrammes de séquence –, décrit l'architecture technique et le modèle de données relationnel, et enfin présente la conception de l'interface utilisateur.

3.2 Présentation générale du projet

Le projet consiste à développer une application web pour la gestion des soutenances et des jurys au sein de la Faculté des Sciences Ben M'Sick. Le système permet de :

- Planifier les soutenances individuelles au sein de sessions globales définies par l'administration.
- Affecter automatiquement les jurys en respectant les contraintes de charge et de spécialité.
- Détecter les conflits horaires, de salle et de double affectation.
- Générer les convocations et documents administratifs.
- Gérer les évaluations et produire les résultats consolidés.

3.3 Cahier des charges

3.3.1 Acteurs du système

Le système doit servir quatre catégories d'acteurs distinctes, chacune disposant de droits d'accès et de fonctionnalités spécifiques :

- **Administrateur** : gère les comptes utilisateurs, configure les paramètres généraux et supervise le bon fonctionnement du système.
- **Coordinateur** : acteur central de l'organisation ; planifie les soutenances dans une session globale, affecte les jurys, valide les plannings et consulte les rapports de synthèse.
- **Enseignant / Membre de jury** : consulte ses convocations, prend connaissance des informations relatives aux étudiants à évaluer et saisit les notes et observations après chaque soutenance.
- **Étudiant** : consulte la date, l'heure, la salle et la composition du jury pour sa soutenance, et peut télécharger sa convocation.

3.3.2 Besoins fonctionnels

Gestion des utilisateurs. Le système doit permettre la création et la gestion des comptes utilisateurs avec une authentification sécurisée. Les quatre rôles définis – administrateur, coordinateur, enseignant et étudiant – doivent chacun disposer d'un ensemble de droits spécifiques.

Gestion des étudiants. L'application doit permettre l'enregistrement des étudiants avec leurs informations complètes : nom, prénom, code national d'étudiant (CNE), filière, niveau, titre du projet et encadrant. Une fonction d'import depuis un fichier CSV ou Excel est prévue. Chaque étudiant doit être associé à un projet spécifique.

Gestion des enseignants et jurys. Le système doit permettre l'ajout et la gestion des enseignants. Par défaut, tous les enseignants sont considérés comme disponibles pour les jurys, mais ils peuvent déclarer leurs plages d'indisponibilité. La composition d'un jury doit inclure la désignation d'un président, d'un rapporteur, d'un examinateur et, le cas échéant, de l'encadrant de l'étudiant.

Gestion des salles. La gestion des salles comprend l'ajout de nouvelles salles, le renseignement de leur capacité, la consultation de leur disponibilité et l'affectation à une soutenance spécifique.

Planification des soutenances. La planification permet la création des soutenances individuelles au sein d'une session globale définie par l'administration, avec la date, l'heure de début, la durée et la salle. Chaque soutenance doit être associée à un étudiant ou à un groupe, et un planning global doit être consultable selon différents critères.

Affectation des jurys. L'affectation des jurys consiste à associer un ensemble d'enseignants à chaque soutenance. Le système doit vérifier qu'un enseignant n'est pas affecté à deux soutenances simultanées et doit rechercher une répartition équilibrée de la charge.

Gestion des conflits. La détection des conflits est une fonctionnalité critique. Le système doit détecter et signaler – avant validation – les conflits de salle, les conflits d'horaire pour un enseignant, la double planification d'un même étudiant et les doublons de planification.

Génération de documents. Le système doit générer : convocations des étudiants, convocations des jurys, planning général, fiches d'évaluation, listes de présence et procès-verbal simplifié. Les formats supportés sont le PDF, l'impression directe et l'export Excel/CSV.

Saisie des évaluations. Les membres du jury ou le coordinateur doivent pouvoir saisir les notes individuelles, ajouter des remarques, puis consolider le résultat final. La décision finale (admis, admis avec corrections, ajourné) est enregistrée après calcul de la moyenne.

Tableau de bord. Un tableau de bord synthétique doit afficher le nombre total de soutenances, le nombre de jurys mobilisés, les salles utilisées, les soutenances planifiées par jour ainsi que le suivi des soutenances évaluées et restant à évaluer.

3.3.3 Besoins non fonctionnels

Performance. L'affichage d'un planning ne doit pas excéder deux secondes. La recherche d'un utilisateur doit être quasi instantanée. La génération d'un document PDF ne doit pas dépasser cinq secondes dans des conditions normales d'utilisation.

Sécurité. L'accès est protégé par une authentification par identifiant et mot de passe. Les mots de passe sont stockés sous forme de hachage cryptographique (BCrypt). Les fonctionnalités et les données sont strictement restreintes selon le rôle de l'utilisateur connecté.

Fiabilité. Le système doit éviter toute perte de données lors des opérations de création ou de modification. Les erreurs de saisie doivent être signalées par des messages compréhensibles. En cas de défaillance, le système doit pouvoir restaurer son état antérieur sans corruption des données.

Maintenabilité. Le code source doit être structuré, documenté et versionné avec Git. Une architecture modulaire facilitera les évolutions ultérieures.

3.4 Règles de gestion

Règles de gestion (RG):

- **RG1** : Un enseignant ne peut être affecté à deux soutenances ayant des horaires qui se chevauchent.
- **RG2** : Une salle ne peut être réservée pour deux soutenances simultanées.
- **RG3** : Un étudiant ne peut avoir qu'une seule soutenance programmée.
- **RG4** : Un jury doit comporter au minimum trois membres (Président, Rapporteur, Examineur). L'encadrant de l'étudiant peut également en faire partie.
- **RG5** : L'encadrant de l'étudiant ne peut en aucun cas occuper le rôle de président du jury.
- **RG6** : La note finale est calculée comme la moyenne des notes individuelles validées par les membres du jury.

Ces règles sont appliquées automatiquement lors de chaque opération de planification ou d'affectation.

3.5 Modules du système

Dans ce document, le terme *module* désigne une unité fonctionnelle du système, regroupant un ensemble cohérent de fonctionnalités liées à un domaine métier spécifique. L'application est structurée en sept modules fonctionnels couvrant l'ensemble du cycle de gestion des soutenances.

3.5.1 Module 1 : Authentification et gestion des rôles

Ce module gère l'accès sécurisé à la plateforme. Il assure la connexion des utilisateurs, la gestion de leurs profils et l'attribution des droits selon les quatre rôles définis.

3.5.2 Module 2 : Gestion académique

Ce module centralise les informations relatives aux étudiants, aux projets de fin d'études, aux encadrants et aux enseignants.

3.5.3 Module 3 : Planification

Ce module permet de créer et d'organiser les soutenances individuelles au sein d'une session globale : définition des horaires, réservation des salles et visualisation du planning global.

3.5.4 Module 4 : Gestion des jurys

Ce module facilite l'affectation des membres de jury à chaque soutenance, la composition équilibrée des commissions, ainsi que la gestion et la consultation des indisponibilités déclarées par les enseignants.

3.5.5 Module 5 : Évaluation

Dédié au processus d'évaluation, ce module permet la saisie des notes, l'ajout d'observations, la génération des fiches d'évaluation et l'enregistrement des décisions de délibération.

3.5.6 Module 6 : Génération documentaire

Ce module automatise la production des documents administratifs essentiels : convocations, plannings, procès-verbaux et exports de données.

3.5.7 Module 7 : Tableau de bord

Le tableau de bord fournit une vue synthétique de l'activité : statistiques sur les soutenances, synthèse des évaluations et suivi en temps réel de l'état d'avancement des sessions.

3.6 Conception UML

Cette section détaille la modélisation UML complète du système. Les diagrammes présentés couvrent l'ensemble des vues nécessaires à la compréhension du système : vue fonctionnelle (cas d'utilisation), vue statique (classes), vue dynamique (séquences, activités, états).

3.6.1 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation permettent de délimiter le périmètre du projet en identifiant les interactions entre les différents acteurs et les fonctionnalités offertes par l'application. Chaque module répond à des objectifs spécifiques, garantissant une séparation claire des responsabilités.

Remarque: Par souci de clarté visuelle, le cas d'utilisation « S'authentifier » est omis des diagrammes détaillés mais reste une pré-condition transversale à toutes les actions du système.

Hiérarchie des acteurs. Le système distingue plusieurs profils d'utilisateurs, chacun possédant des droits et des responsabilités spécifiques. La hiérarchie permet d'hériter des fonctionnalités communes, notamment la consultation du planning et la gestion du profil personnel.

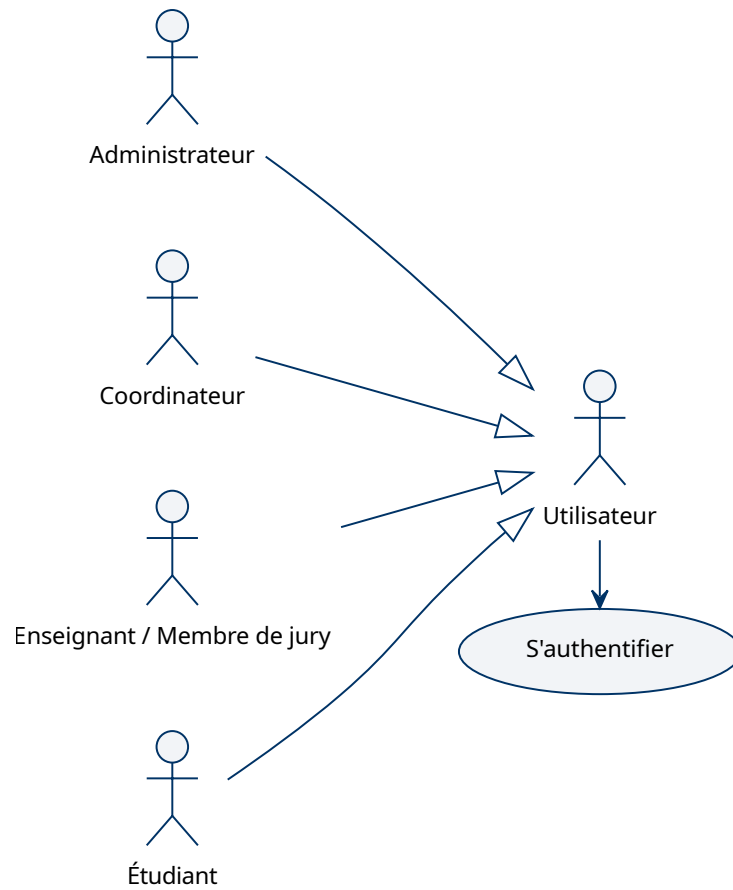


FIGURE 4 – Hiérarchie générale des acteurs

Module Administration. Ce module est réservé à la gestion technique et structurelle de la plateforme. L'administrateur veille au bon fonctionnement du système en gérant les ressources de base.

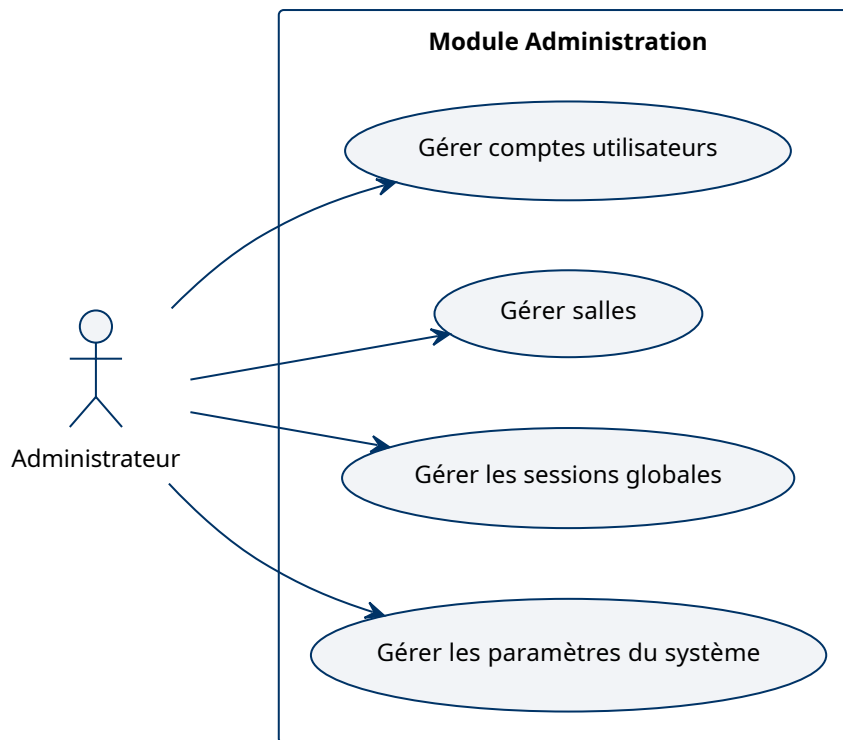


FIGURE 5 – Cas d'utilisation : Administration du système

Cas d'utilisation	Description
Gérer comptes utilisateurs	Création, modification (incluant rôles et mots de passe) et suppression des comptes.
Gérer salles	Administration du catalogue des salles, capacité et disponibilités.
Gérer sessions globales	Création, modification et clôture des sessions globales (normale, rattrapage).
Gérer paramètres système	Configuration des paramètres transversaux : durées par défaut, périodes, règles générales.

TABLE 5 – Description des cas d'utilisation – Administrateur

Module Coordination. Le coordinateur est l'acteur central du processus de planification. Dans ce module, le terme *soutenance* désigne une séance individuelle planifiée pour un étudiant et son jury, tandis que la *session globale* désigne le cadre administratif créé par l'administrateur. Le coordinateur travaille sur les soutenances contenues dans une session globale, de l'importation massive des données à la validation finale du planning.

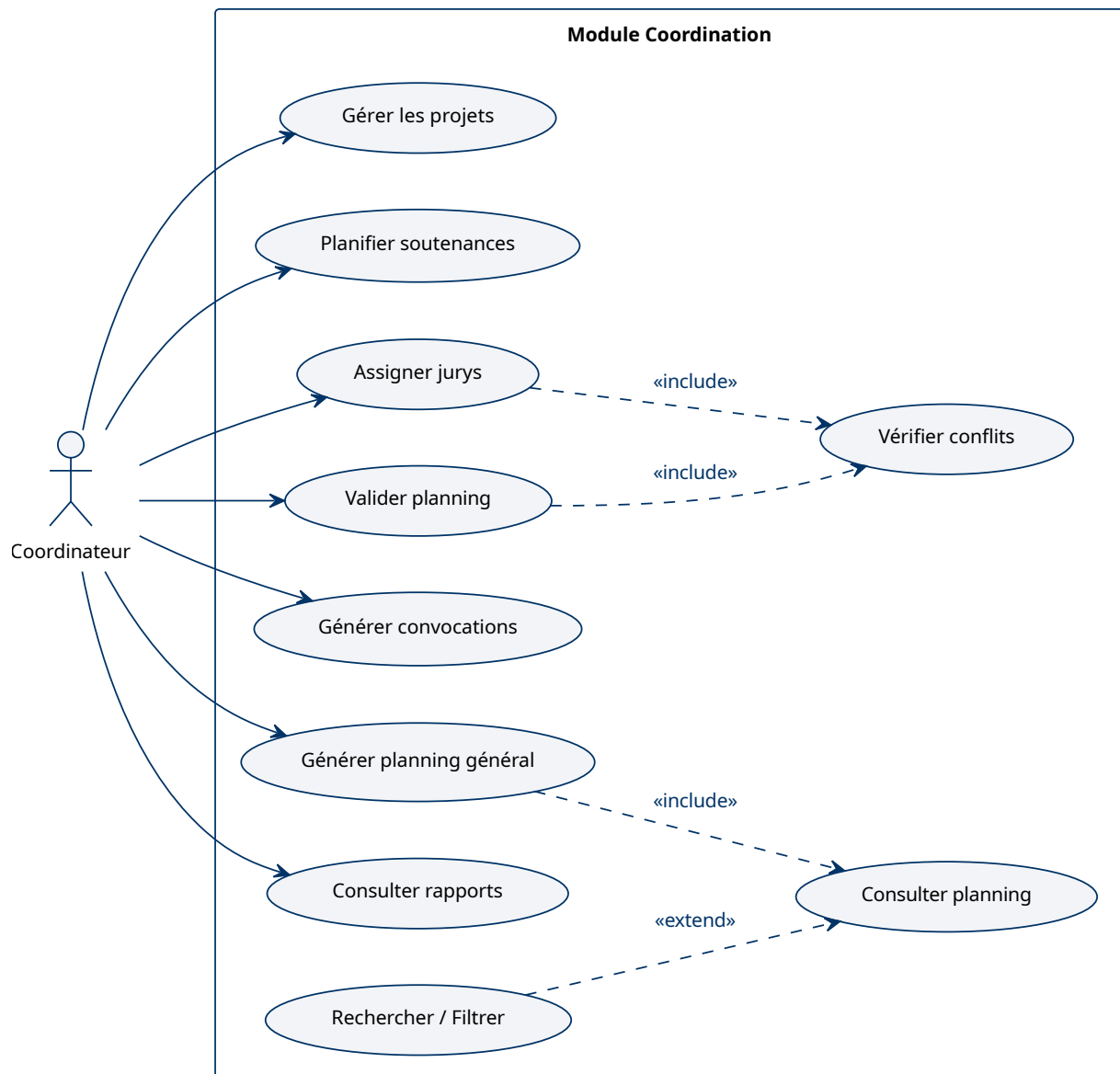


FIGURE 6 – Cas d'utilisation : Coordination des soutenances

Cas d'utilisation	Description
Assigner jurys	Constitution des commissions (président, rapporteur, examinateur).
Planifier soutenances	Planification des créneaux, définition des horaires et affectation des salles.
Valider planning	Confirmation finale du planning après traitement des alertes et conflits.
Gérer projets	Gestion des thèmes, titres de PFE et association étudiants-projets-encadrants.
Consulter planning	Visualisation globale filtrable (date, salle, enseignant, filière, étudiant).
Rechercher / Filtrer	Recherche multicritère avancée pour la supervision.
Générer convocations	Production automatique des documents PDF pour étudiants et jurys.
Générer planning général	Production du planning complet et liste de présence pour la session.
Consulter rapports	Accès aux synthèses, statistiques et procès-verbaux simplifiés.
Vérifier conflits	Détection des chevauchements, doubles planifications et doublons.

TABLE 6 – Description des cas d'utilisation – Coordinateur

Module Jury. Les enseignants interagissent avec le système principalement pour consulter leurs obligations de jury et évaluer les travaux des étudiants.

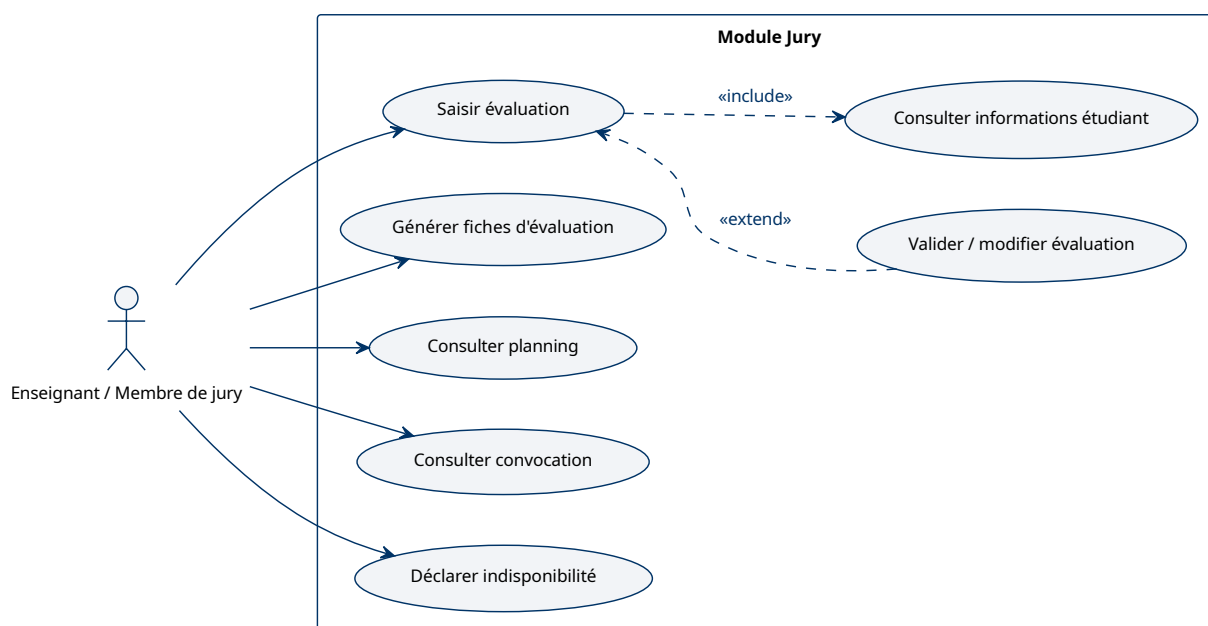


FIGURE 7 – Cas d'utilisation : Enseignant et Jury

Cas d'utilisation	Description
Consulter planning	Accès au planning personnel des soutenances.
Consulter convocation	Visualisation et téléchargement des convocations.
Saisir évaluation	Enregistrement des notes et observations pendant ou après la soutenance.
Valider / modifier évaluation	Finalisation de la notation et modification des remarques.
Consulter informations étudiant	Consultation des détails du projet et du dossier.
Générer fiches d'évaluation	Création des supports de notation pour la session.
Déclarer indisponibilité	Signalement des créneaux horaires non disponibles.

TABLE 7 – Description des cas d'utilisation – Enseignant

Module Étudiant. L'interface dédiée à l'étudiant est conçue pour être simple et directe. Elle lui permet de se concentrer sur les informations essentielles liées à sa soutenance.

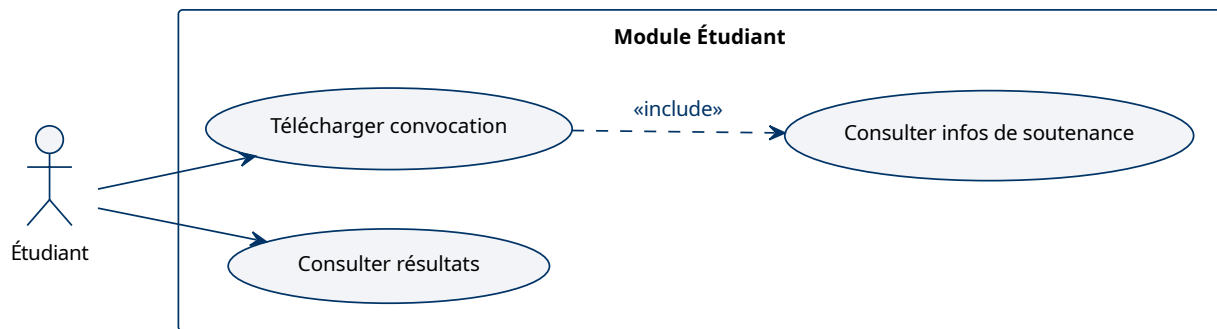


FIGURE 8 – Cas d'utilisation : Espace Étudiant

Cas d'utilisation	Description
Télécharger convocation	Récupération du document PDF officiel.
Consulter infos de soutenance	Visualisation de la date, l'heure, la salle et le jury.
Consulter résultats	Consultation des notes et de la décision finale.

TABLE 8 – Description des cas d'utilisation – Étudiant

3.6.2 Diagramme de classes

Le diagramme de classes constitue le pivot de la conception statique. Il définit la structure de données interne et les règles métier qui régissent les interactions entre les objets du système.

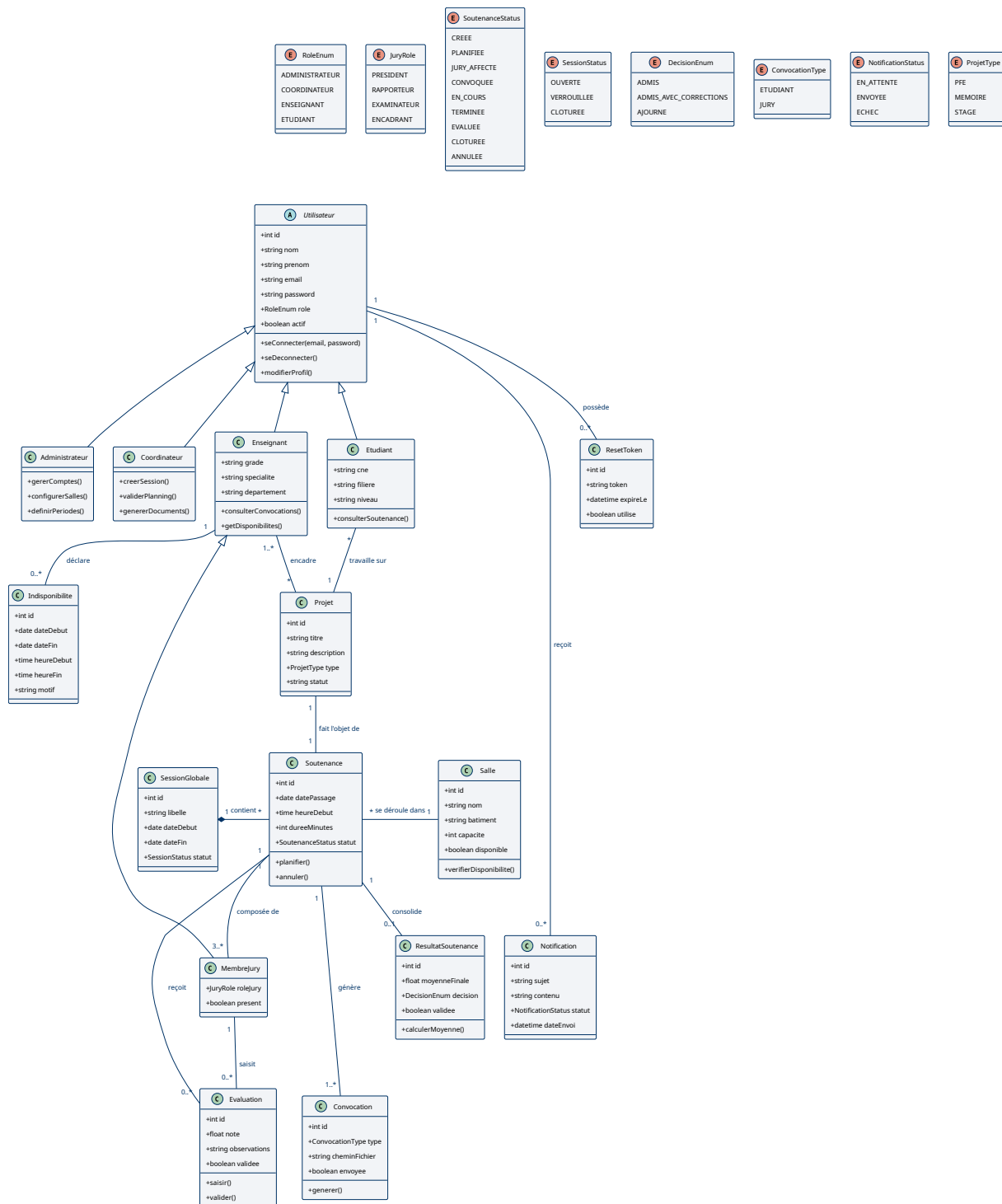


FIGURE 9 – Diagramme de classes global du système

Les entités clés identifiées sont les suivantes :

- **Utilisateur** : Classe de base gérant l'identité, les identifiants de connexion et les droits d'accès (rôles).
- **Soutenance** : Entité centrale faisant le lien entre un projet, un jury, une salle et un créneau temporel au sein d'une session globale.

- **Projet** : Représente le travail académique (PFE, Mémoire, Stage), supervisé par un encadrant.
- **Jury** : Structure d'association définissant les rôles (Président, Rapporteur, Examineur).
- **Évaluation et résultat** : Chaque membre du jury saisit une note individuelle ; le résultat consolide ces notes en moyenne finale.

3.6.3 Cycle de vie d'une soutenance

Le diagramme d'états ci-dessous illustre les transitions de l'entité Soutenance. Ce cycle garantit que les actions (saisie des notes, génération des convocations) ne sont autorisées que lorsque la soutenance est dans l'état approprié.

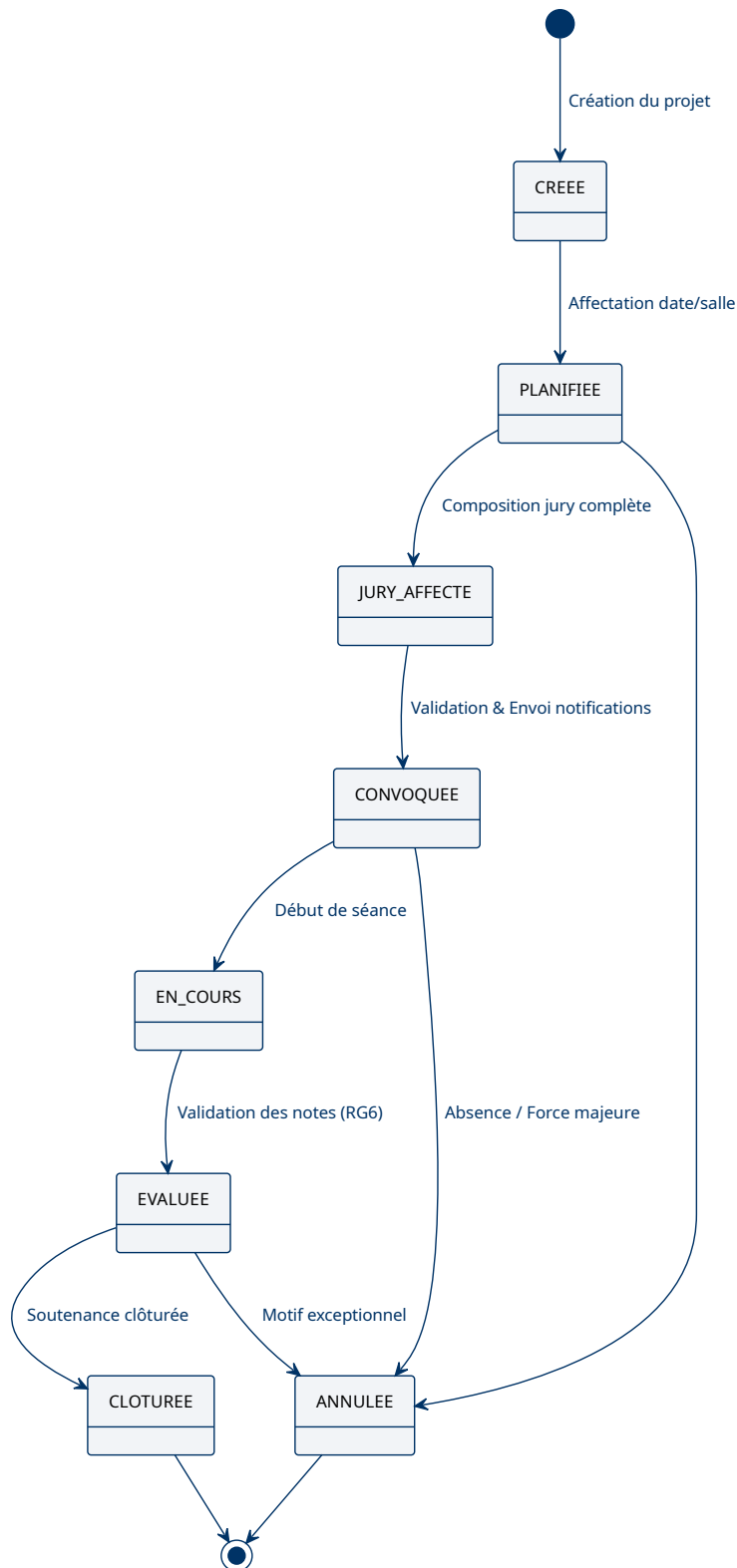


FIGURE 10 – Diagramme d'états : Cycle de vie d'une soutenance

3.6.4 Diagrammes d'activité

Les diagrammes d'activité détaillent la logique algorithmique et le flux de contrôle des processus métier critiques.

Workflow de planification. Ce processus décrit les étapes de planification des soutenances à l'intérieur d'une session globale. Le workflow est itératif, permettant au coordinateur de corriger les anomalies détectées (conflits de salle, indisponibilité) avant la validation finale.

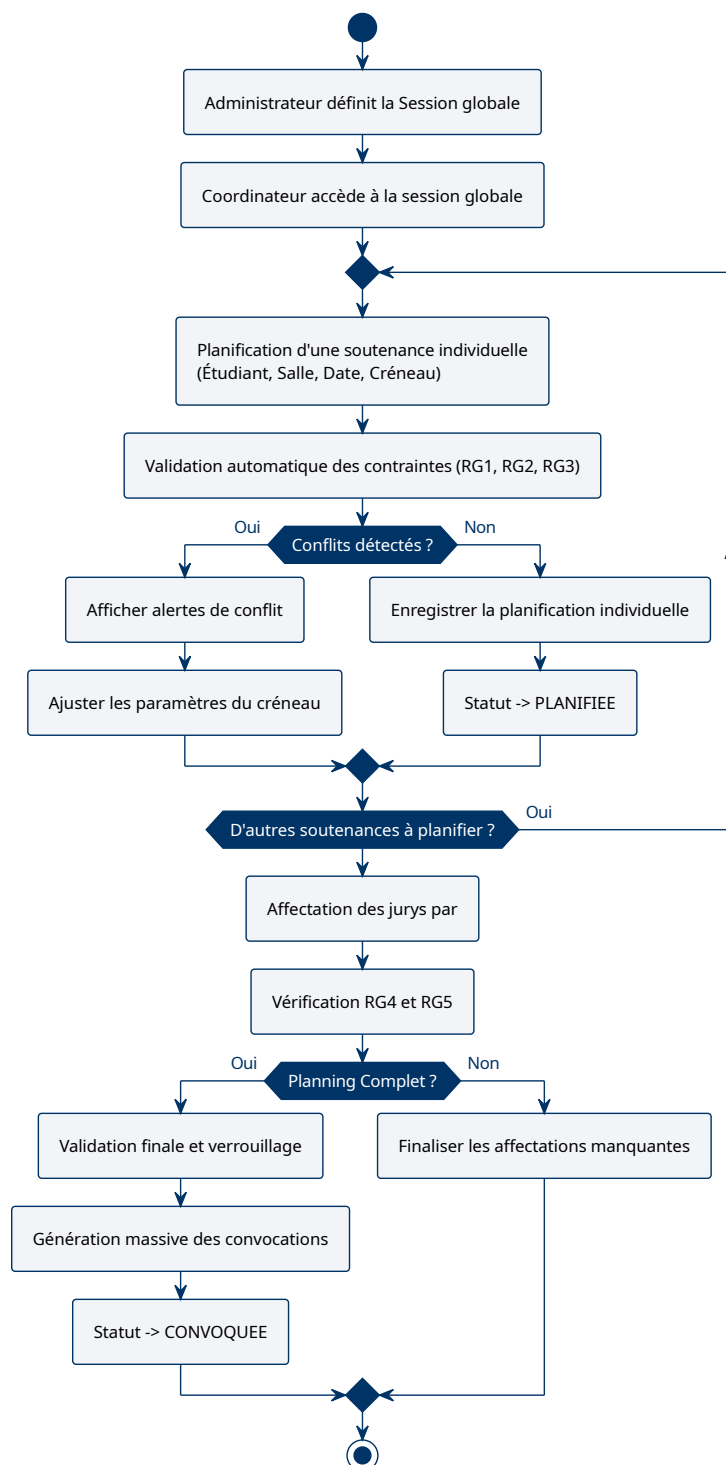


FIGURE 11 – Diagramme d'activité : Planification de session

Déroulement des évaluations. Ce diagramme illustre le processus d'évaluation, du contrôle de présence jusqu'au calcul automatique de la moyenne finale.

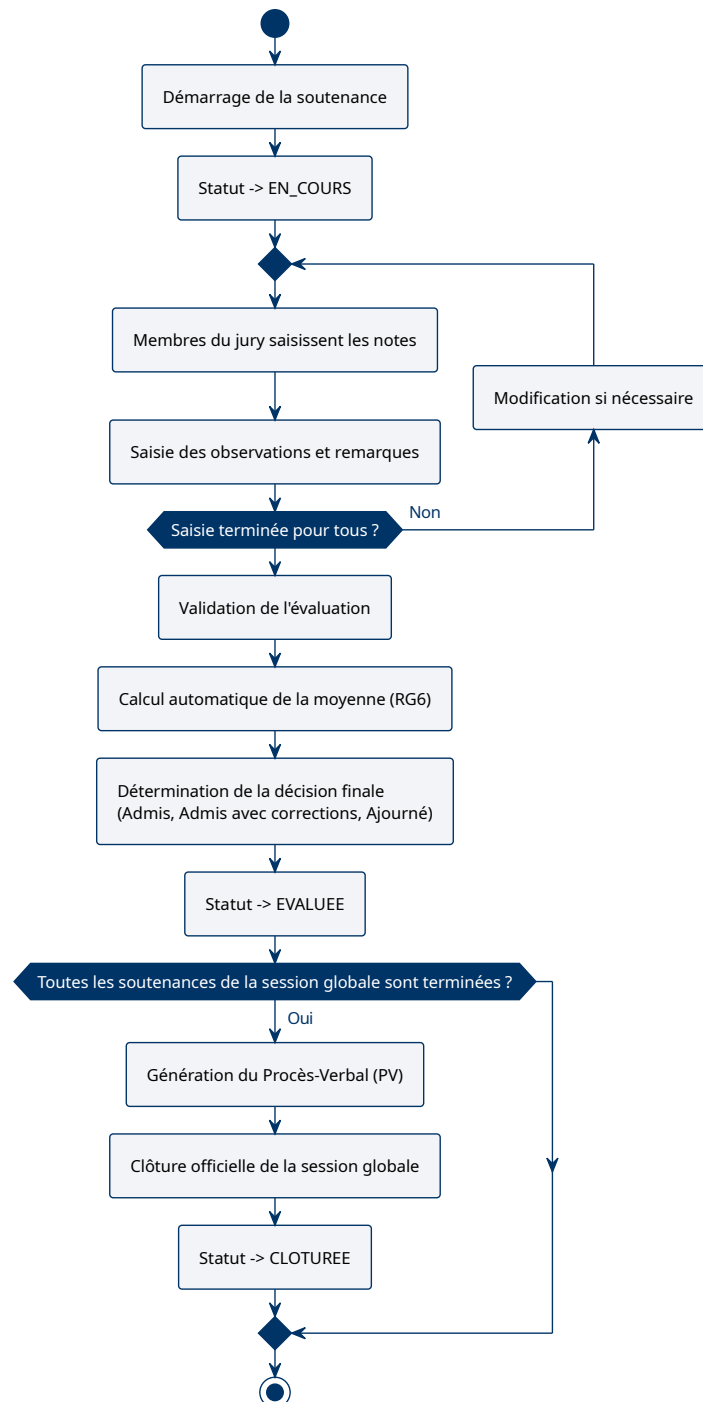


FIGURE 12 – Diagramme d'activité : Processus d'évaluation

3.6.5 Diagrammes de séquence

Les diagrammes de séquence détaillent les interactions chronologiques entre les composants (Client, Contrôleur, Services, Modèle) lors de l'exécution des processus métier.

Processus d'authentification. L'authentification repose sur un échange sécurisé où le contrôleur valide les identifiants via le service utilisateur avant de générer un token JWT.

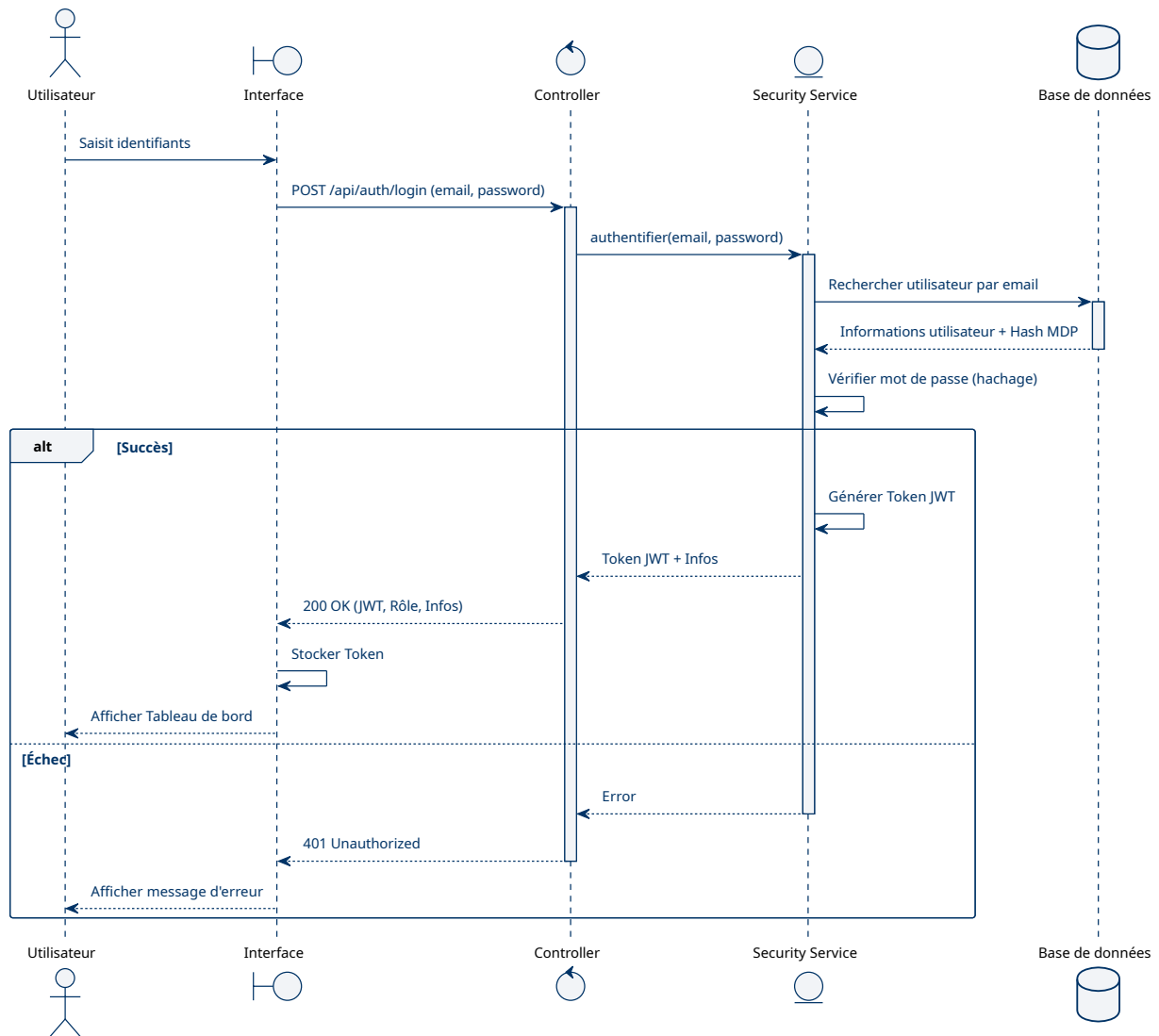


FIGURE 13 – Diagramme de séquence : Authentification

Planification et détection de conflits. Lors de la planification, le PlanningService sollicite le ConflictService pour vérifier l'absence de chevauchements.

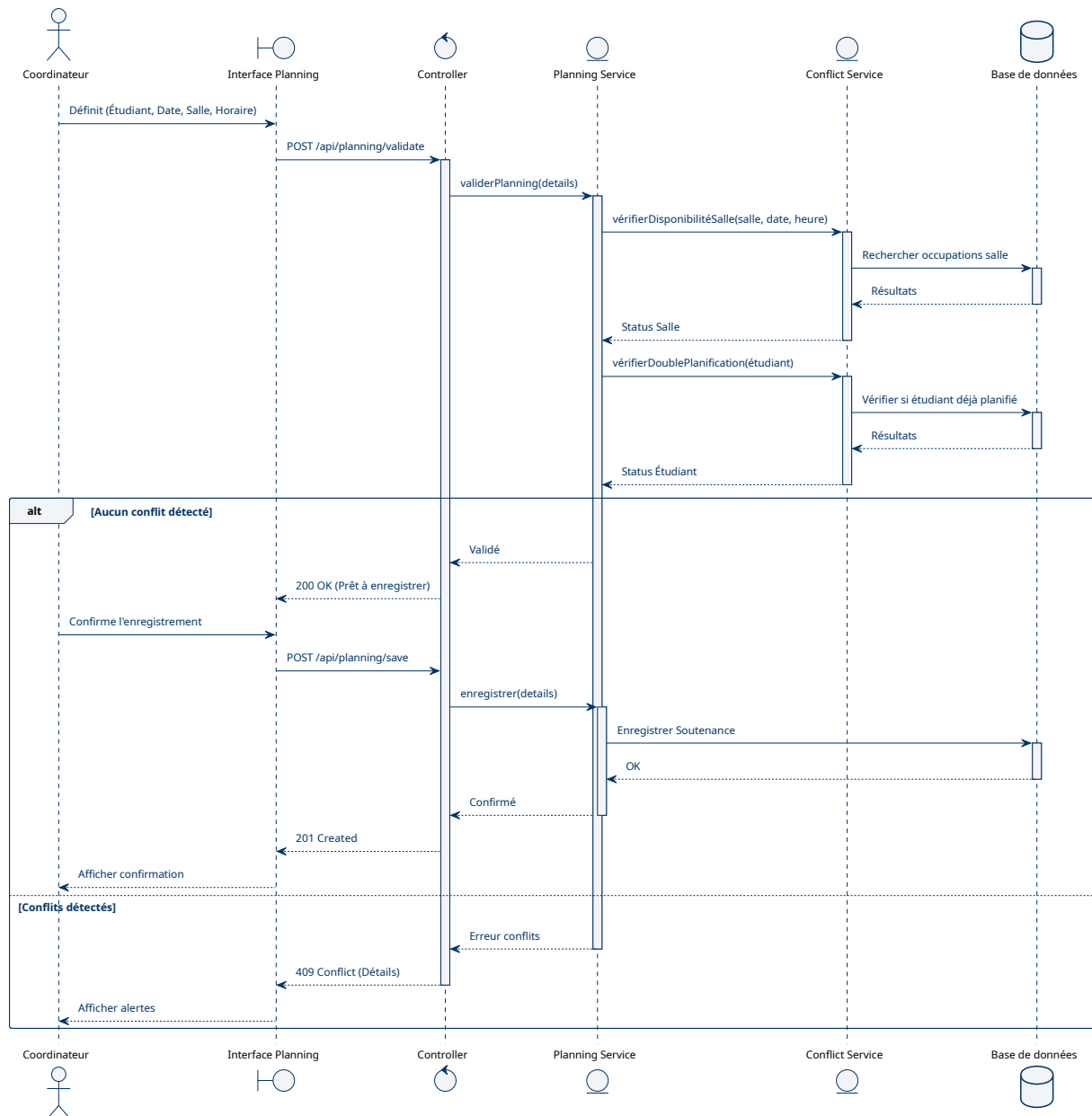


FIGURE 14 – Diagramme de séquence : Planification de soutenance

Affectation et validation du jury. L'affectation des jurys illustre les règles métier RG4 et RG5. Le service métier coordonne la sélection des enseignants selon la charge de travail et la spécialité.

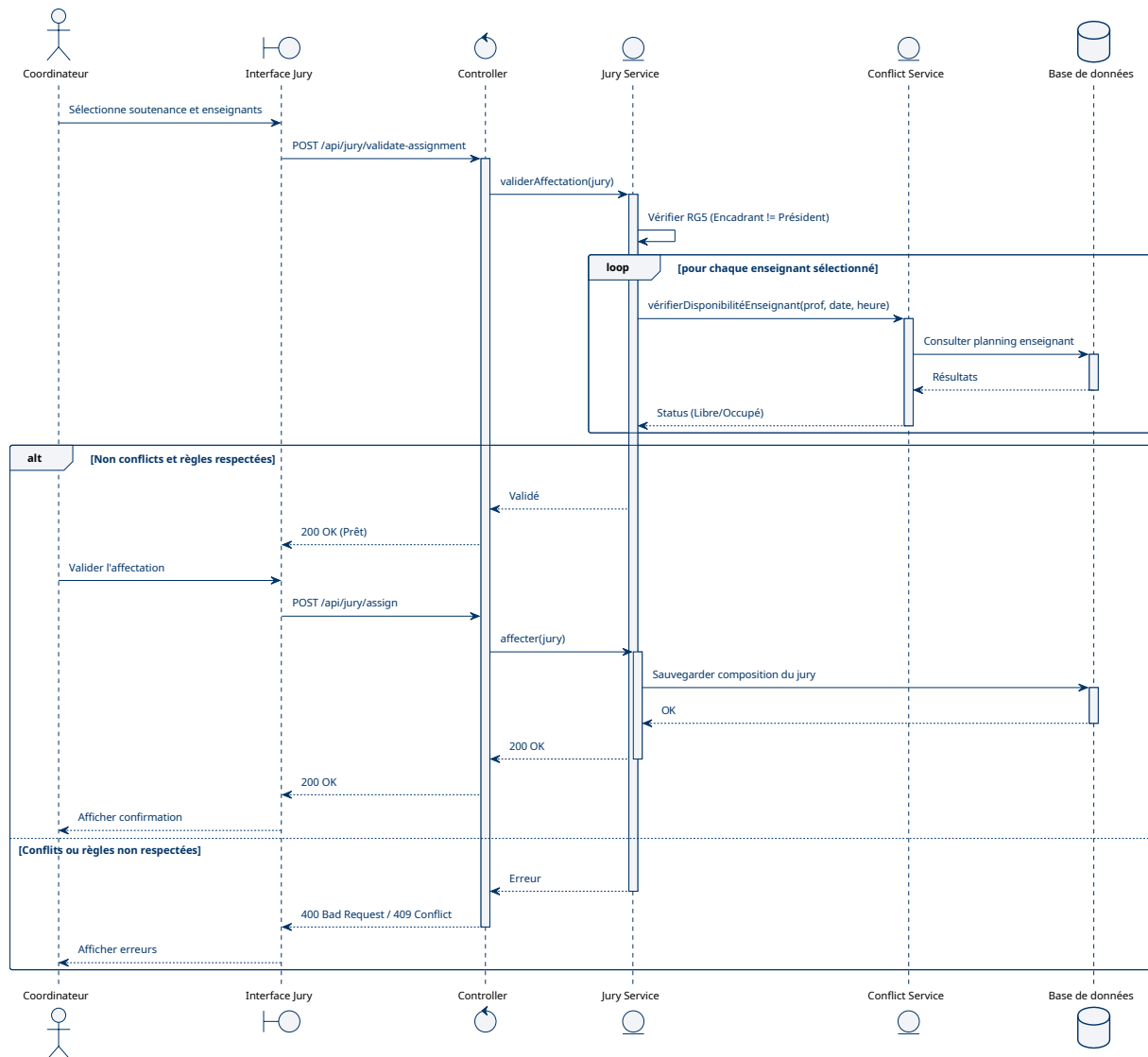


FIGURE 15 – Diagramme de séquence : Affectation du jury

Saisie et validation d'une évaluation. Ce flux montre comment les membres du jury enregistrent leurs notes. Le système recalcule la moyenne provisoire à chaque saisie et ne finalise le résultat que lorsque toutes les notes sont validées.

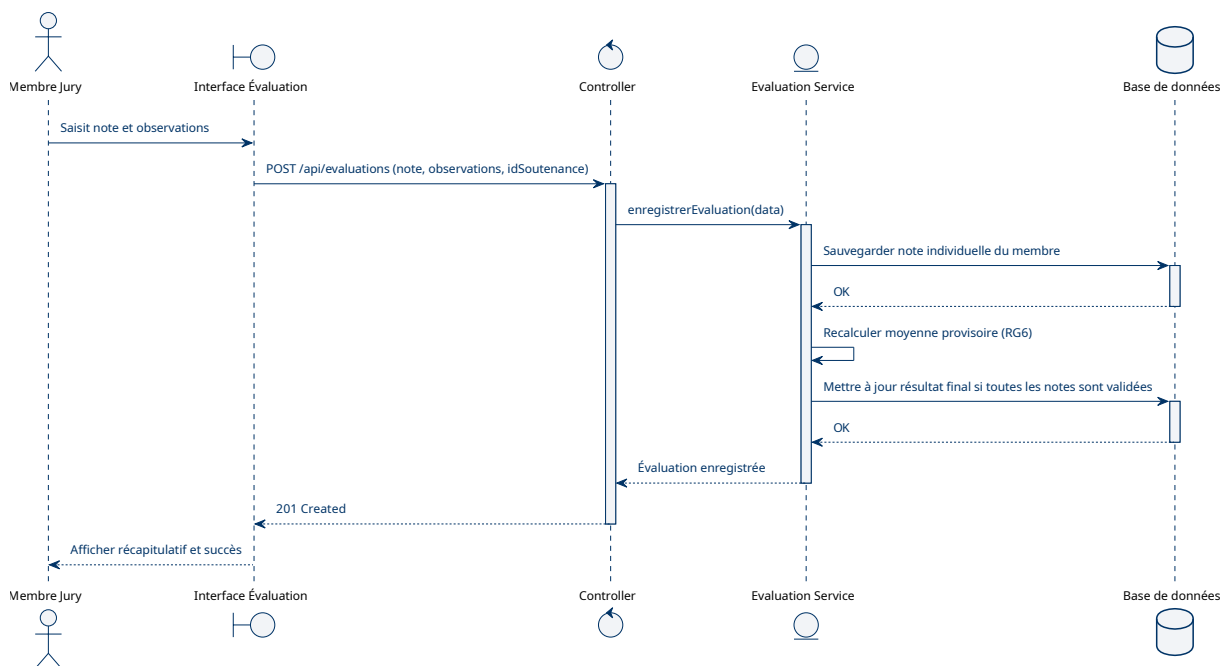


FIGURE 16 – Diagramme de séquence : Saisie d'évaluation

Génération PDF. Le service de documents extrait les données consolidées du modèle pour les injecter dans un template normalisé.

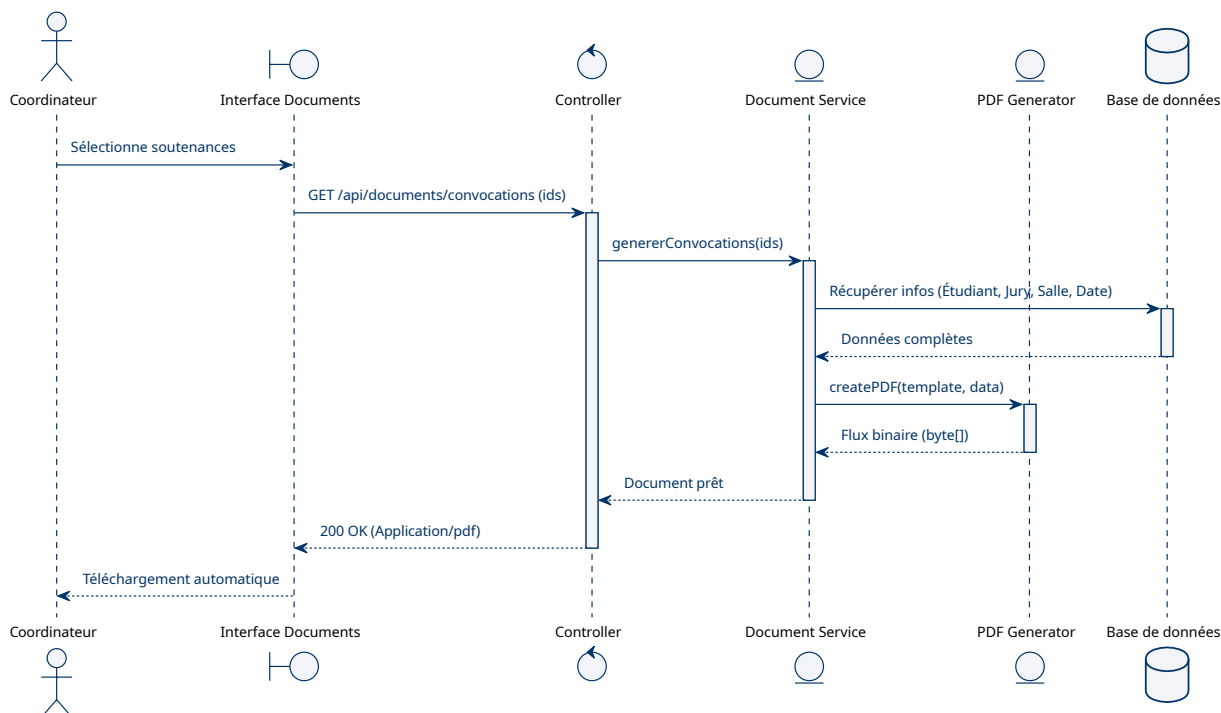


FIGURE 17 – Diagramme de séquence : Génération PDF

Import massif de données. Le processus d'importation gère la lecture de fichiers externes, effectue une validation ligne par ligne et assure la persistance transactionnelle.

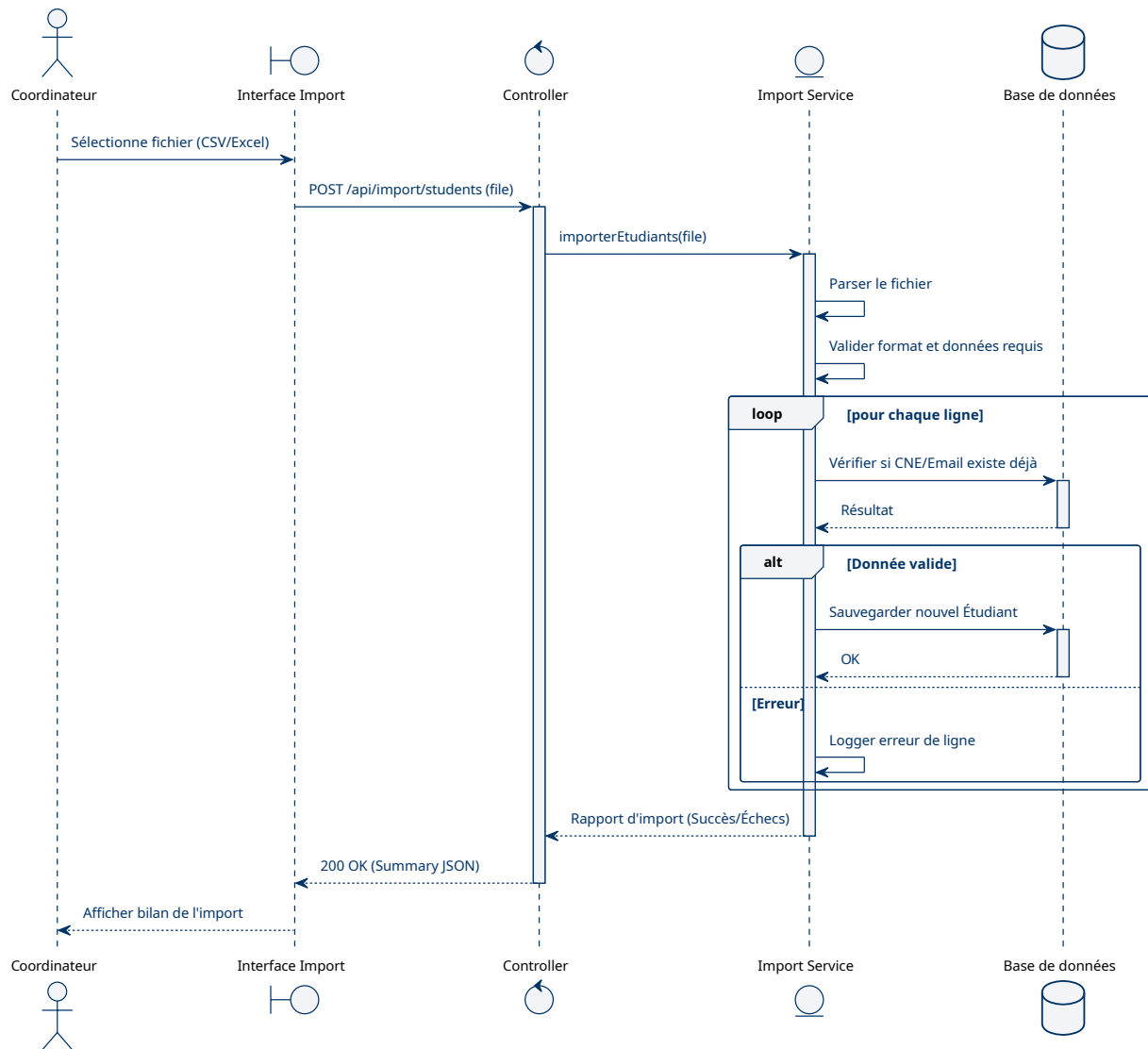


FIGURE 18 – Diagramme de séquence : Import massif de données

Validation finale et notifications. La validation par le coordinateur verrouille le planning et déclenche des événements asynchrones (génération et envoi des notifications).

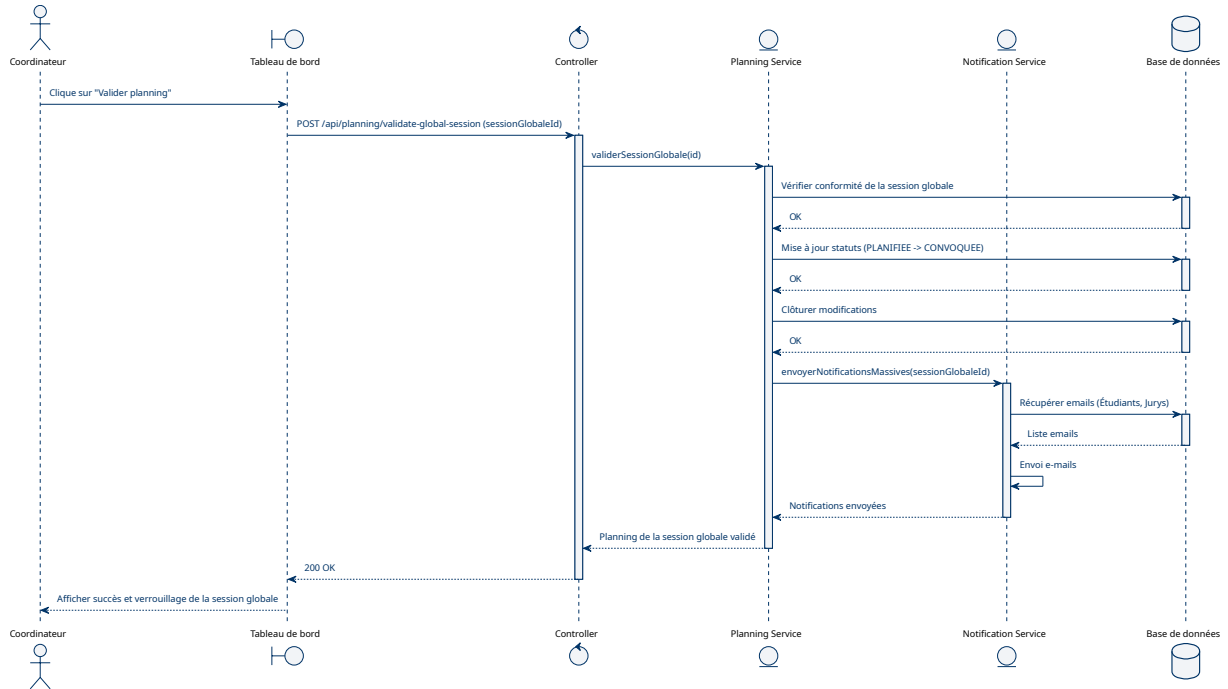


FIGURE 19 – Diagramme de séquence : Validation finale et notifications

Récupération de compte. En cas d'oubli du mot de passe, un lien temporaire contenant un jeton unique est envoyé pour une réinitialisation sécurisée.

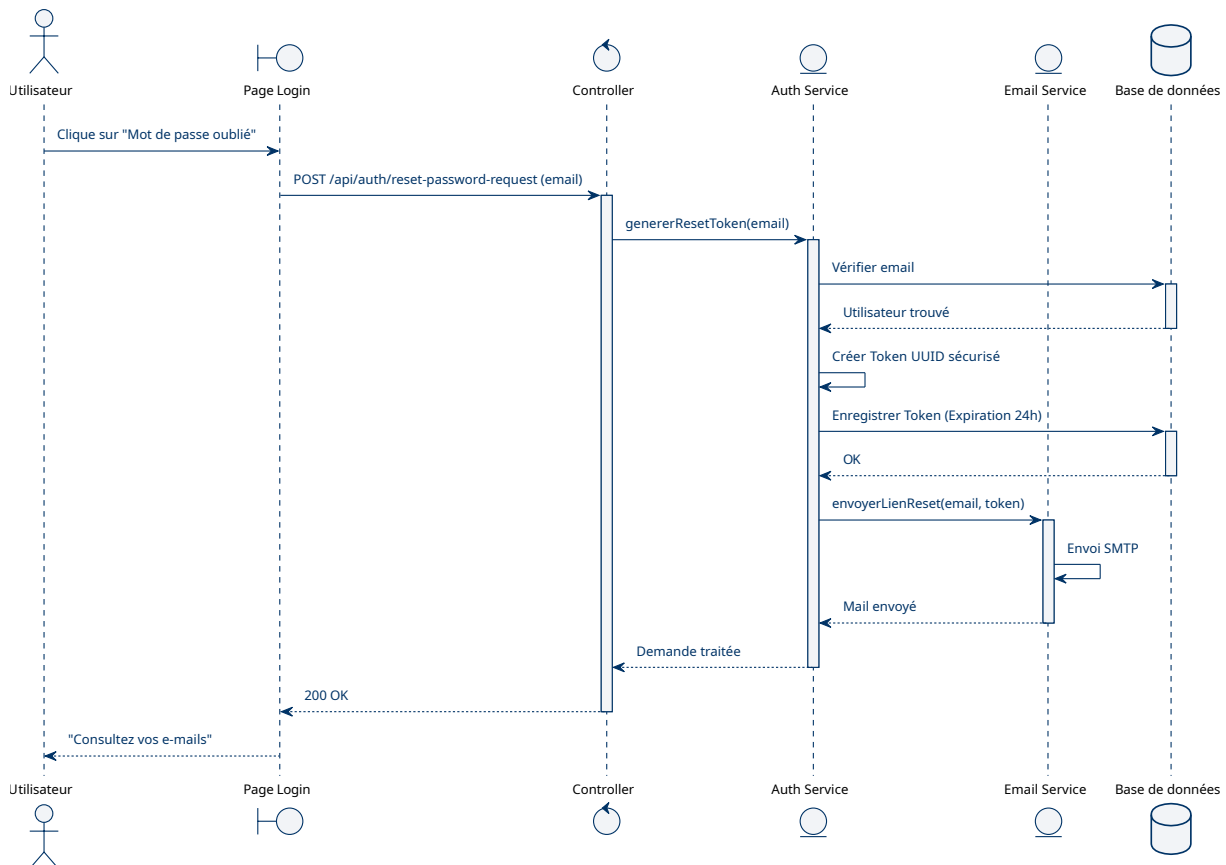


FIGURE 20 – Diagramme de séquence : Réinitialisation de mot de passe

3.7 Architecture technique

Le système repose sur une architecture web moderne organisée en couches, garantissant une séparation claire des responsabilités, une sécurité renforcée et une maintenance facilitée.

3.7.1 Architecture en couches

L'application suit le pattern **MVC (Modèle-Vue-Contrôleur)** adapté au Web :

- **Vue (View)** : Assurée par le Frontend React, elle gère l'interface utilisateur et l'expérience interactive de manière asynchrone.
- **Contrôleur (Controller)** : Les contrôleurs Spring Boot exposent des endpoints REST, reçoivent les requêtes du client et orchestrent les réponses.
- **Modèle (Model)** : Comprend la logique métier (Services), l'accès aux données (Repositories) et la structure de persistance MySQL.

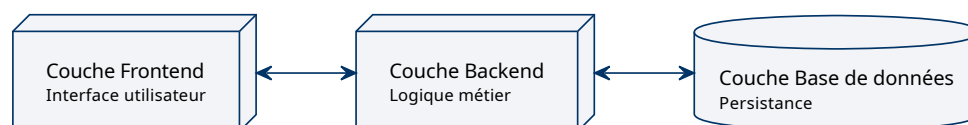


FIGURE 21 – Architecture générale en trois couches

3.7.2 Architecture technique des composants

Le diagramme suivant détaille l'organisation interne des modules et leur mode de communication. On y observe la séparation nette entre le client web et le serveur d'application qui encapsule la logique métier complexe.

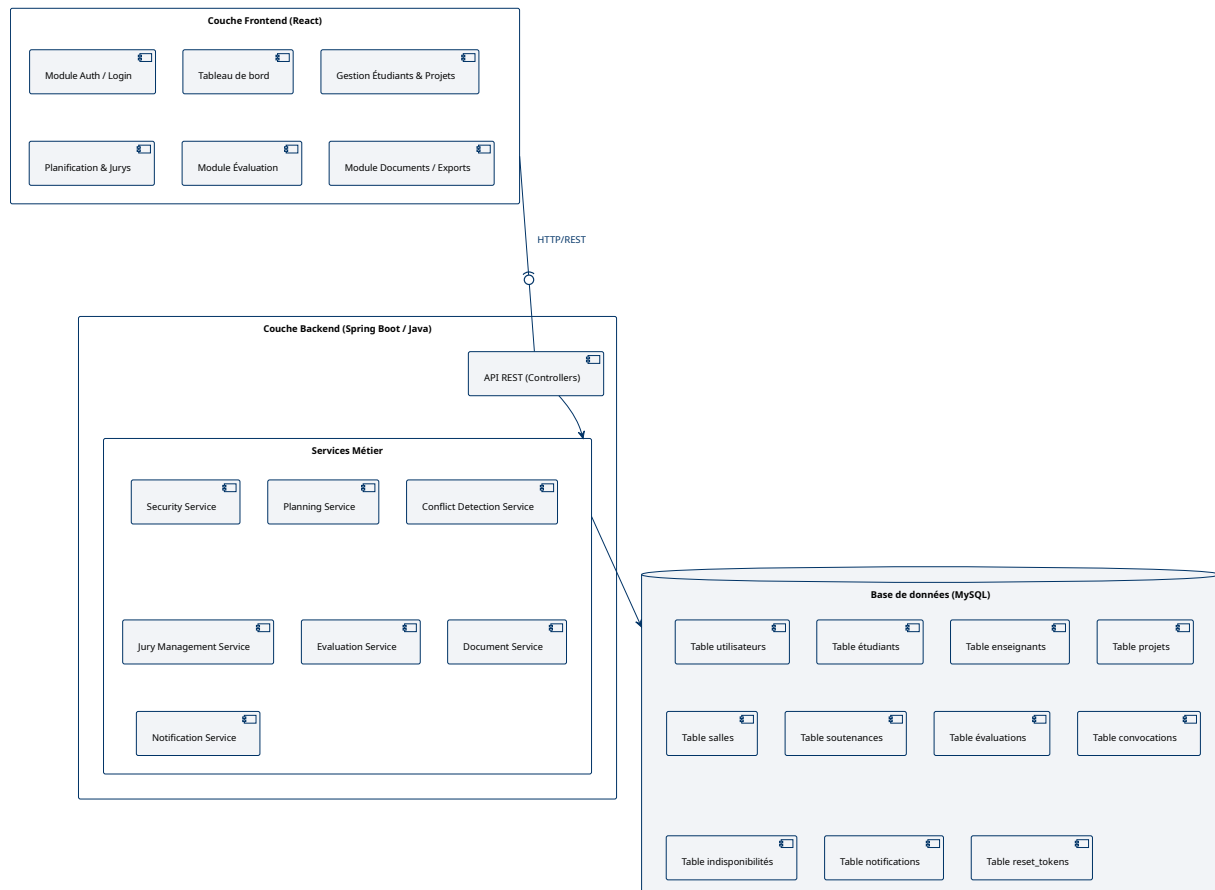


FIGURE 22 – Architecture technique des composants

3.7.3 Sécurité et Authentification JWT

Pour répondre aux exigences de sécurité et de performance, le système implémente une authentification **stateless** basée sur les **JSON Web Tokens (JWT)** :

- **Génération** : Lors de la connexion, le serveur valide les identifiants et génère un token signé cryptographiquement contenant l'identité et le rôle de l'utilisateur.
- **Transmission** : Ce token est renvoyé au client (React) qui le stocke localement et l'inclut dans le header **Authorization** de chaque requête API ultérieure.
- **Validation** : Le backend (Spring Security) intercepte chaque requête, vérifie la signature du token et extrait les droits d'accès sans avoir à interroger la base de données pour la session.

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant unique de l'utilisateur.
nom, prenom	VARCHAR	NOT NULL	Informations civiles.
email	VARCHAR	UNIQUE	Adresse de contact et identifiant de connexion.
password	VARCHAR	NOT NULL	Hash du mot de passe (BCrypt).
role	ENUM	NOT NULL	Rôle (ADMIN, COORDINATEUR, ENSEIGNANT, ETUDIANT).
actif	BOOLEAN	DEFAULT TRUE	État d'activation du compte.

TABLE 9 – Dictionnaire – Table utilisateurs

Champ	Type	Contrainte	Description
cne	VARCHAR	PK	Code National de l'Étudiant.
filiere	VARCHAR	NOT NULL	Nom de la filière d'attachement.
niveau	VARCHAR	NOT NULL	Niveau académique (ex : Licence, Master).
user_id	INT	FK	Référence vers le compte utilisateur.
projet_id	INT	FK	Référence vers le projet porté par l'étudiant.

TABLE 10 – Dictionnaire – Table etudiants

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant unique de l'enseignant.
grade	VARCHAR	-	Grade académique (ex : PES, PH).
specialite	VARCHAR	-	Domaine d'expertise.
user_id	INT	FK	Référence vers le compte utilisateur.

TABLE 11 – Dictionnaire – Table enseignants

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant unique du profil spécifique.
user_id	INT	FK	Référence vers le compte utilisateur.

TABLE 12 – Dictionnaire – Table administrateurs / coordinateurs

Gestion des utilisateurs et des rôles.

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant de la session globale.
libelle	VARCHAR	NOT NULL	Nom de la session (normale, rattrapage).
date_debut	DATE	NOT NULL	Début de la période de soutenances.
date_fin	DATE	NOT NULL	Fin de la période de soutenances.
statut	ENUM	NOT NULL	État (OUVERTE, VERROUILLEE, CLOTUREE).

TABLE 13 – Dictionnaire – Table sessions_globales

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant unique du projet.
titre	VARCHAR	NOT NULL	Intitulé du sujet de PFE/Mémoire.
description	TEXT	-	Résumé ou détails du projet.
type	ENUM	NOT NULL	Type (PFE, MEMOIRE, STAGE).
encadrant_id	INT	FK	Référence vers l'enseignant encadrant.

TABLE 14 – Dictionnaire – Table projets

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant unique de la salle.
nom	VARCHAR	NOT NULL	Nom ou numéro de la salle.
batiment	VARCHAR	-	Localisation géographique.
capacite	INT	-	Nombre de places assises.

TABLE 15 – Dictionnaire – Table salles

Organisation académique et logistique.

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant unique de la soutenance.
date_passage	DATE	NOT NULL	Date prévue pour la séance.
heure_debut	TIME	NOT NULL	Heure précise du début.
duree	INT	-	Durée prévue en minutes.
statut	ENUM	NOT NULL	État (PLANIFIEE, CONVOQUEE, EVALUEE...).
salle_id	INT	FK	Référence à la salle occupée.
projet_id	INT	FK	Référence au projet présenté.
session_id	INT	FK	Référence à la session globale.

TABLE 16 – Dictionnaire – Table soutenances

Champ	Type	Contrainte	Description
soutenance_id	INT	PK, FK	Référence à la soutenance.
enseignant_id	INT	PK, FK	Référence à l'enseignant membre.
role_jury	ENUM	NOT NULL	Rôle (PRESIDENT, RAPPORTEUR, EXAMINATEUR).
present	BOOLEAN	-	Confirmation de présence.

TABLE 17 – Dictionnaire – Table membres_jury

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant de la note individuelle.
note	FLOAT	NOT NULL	Note attribuée par un membre du jury.
observations	TEXT	-	Remarques du membre évaluateur.
validee	BOOLEAN	DEFAULT FALSE	Note confirmée.
soutenance_id	INT	FK	Référence à la soutenance évaluée.
enseignant_id	INT	FK	Membre du jury ayant saisi la note.

TABLE 18 – Dictionnaire – Table evaluations

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant du résultat consolidé.
moyenne_finale	FLOAT	NOT NULL	Moyenne des notes validées (RG6).
decision	ENUM	NOT NULL	Décision finale (ADMIS, AJOURNE...).
observations_finales	TEXT	-	Remarques finales du jury.
validee	BOOLEAN	DEFAULT FALSE	Verrouillage du résultat.
soutenance_id	INT	FK, UNIQUE	Soutenance concernée.

TABLE 19 – Dictionnaire – Table resultats_soutenance

Soutenances et Évaluations. La table evaluations conserve la traçabilité des notes par membre du jury. La table resultats_soutenance porte le résultat unique affiché à l'étudiant.

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant de la convocation.
type	ENUM	NOT NULL	Type (ETUDIANT ou JURY).
chemin_fichier	VARCHAR	-	Emplacement du PDF généré.
envoyee	BOOLEAN	DEFAULT FALSE	Envoi effectué.
date_generation	DATETIME	NOT NULL	Date de génération.
soutenance_id	INT	FK	Soutenance concernée.
utilisateur_id	INT	FK	Destinataire.

TABLE 20 – Dictionnaire – Table convocations

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant de l'indisponibilité.
date_debut	DATE	NOT NULL	Premier jour indisponible.
date_fin	DATE	NOT NULL	Dernier jour indisponible.
heure_debut	TIME	-	Début du créneau si partiel.
heure_fin	TIME	-	Fin du créneau si partiel.
motif	VARCHAR	-	Motif ou commentaire.
enseignant_id	INT	FK	Enseignant concerné.

TABLE 21 – Dictionnaire – Table indisponibilites

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant de la notification.
sujet	VARCHAR	NOT NULL	Objet du message.
contenu	TEXT	NOT NULL	Corps du message.
statut	ENUM	NOT NULL	EN_ATTENTE, ENVOYEE ou ECHEC.
date_envoi	DATETIME	-	Date effective d'envoi.
utilisateur_id	INT	FK	Destinataire.
soutenance_id	INT	FK	Soutenance liée.

TABLE 22 – Dictionnaire – Table notifications

Champ	Type	Contrainte	Description
id	INT	PK, AI	Identifiant du jeton.
token	VARCHAR	UNIQUE, NOT NULL	Valeur aléatoire du jeton sécurisé.
expire_le	DATETIME	NOT NULL	Date limite d'utilisation.
utilise	BOOLEAN	DEFAULT FALSE	Empêche la réutilisation.
utilisateur_id	INT	FK	Compte utilisateur concerné.

TABLE 23 – Dictionnaire – Table reset_tokens

Documents, disponibilités et sécurité opérationnelle.

3.9 Conception de l'interface utilisateur

3.9.1 Plan de navigation

Le diagramme suivant présente le sitemap de l'application et les points d'entrée selon les rôles utilisateurs.

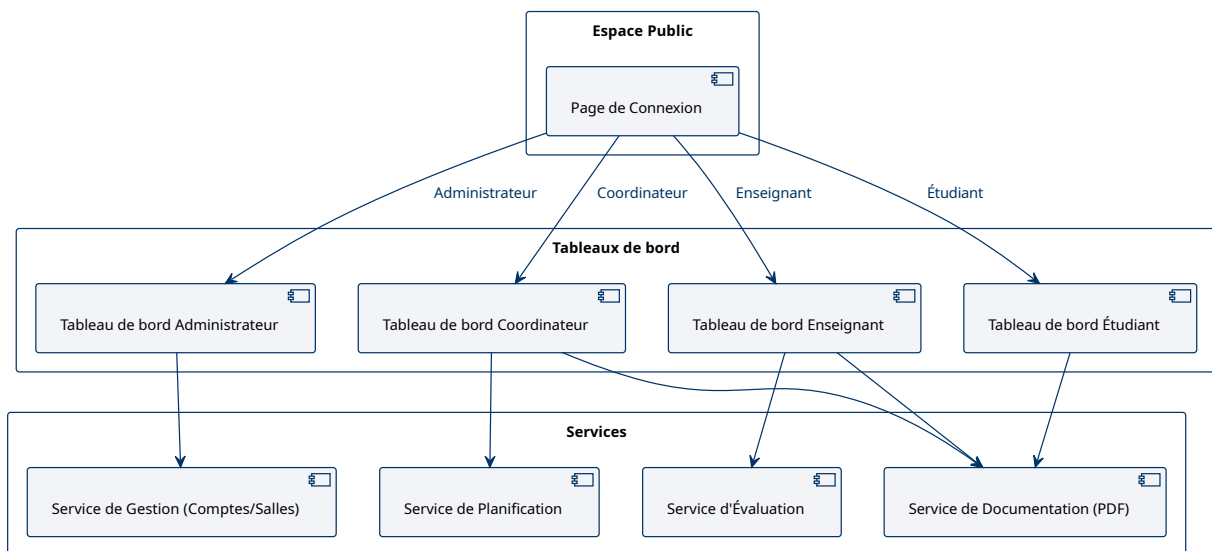


FIGURE 24 – Sitemap et plan de navigation

3.9.2 Prototypes d'interface

Cette section présente les maquettes graphiques des principales interfaces de l'application. Le design privilégie la sobriété, l'efficacité et une navigation intuitive.

Interface d'accès. L'écran de connexion constitue le point d'entrée unique du système. Il assure la sécurité des accès et redirige chaque utilisateur vers son espace dédié selon son rôle.

SG Soutenances
UNIVERSITY MANAGEMENT SYSTEM

L'excellence académique au bout des doigts.

Une plateforme moderne pour la gestion, la planification et le suivi des soutenances universitaires.

Officiel | Sécurisé | Intuitif

Connexion
Choisissez votre profil pour accéder à votre espace

IDENTIFIANTS

Email académique
nom.prenom@univ.dz

Mot de passe

Se connecter

© 2026 Copyright Assistance Mentions Légales

FIGURE 25 – Interface de connexion

Tableau de bord Administrateur. Cette interface offre une vision globale sur l'infrastructure. Elle permet de gérer le cycle de vie des comptes utilisateurs et de configurer les ressources logistiques.

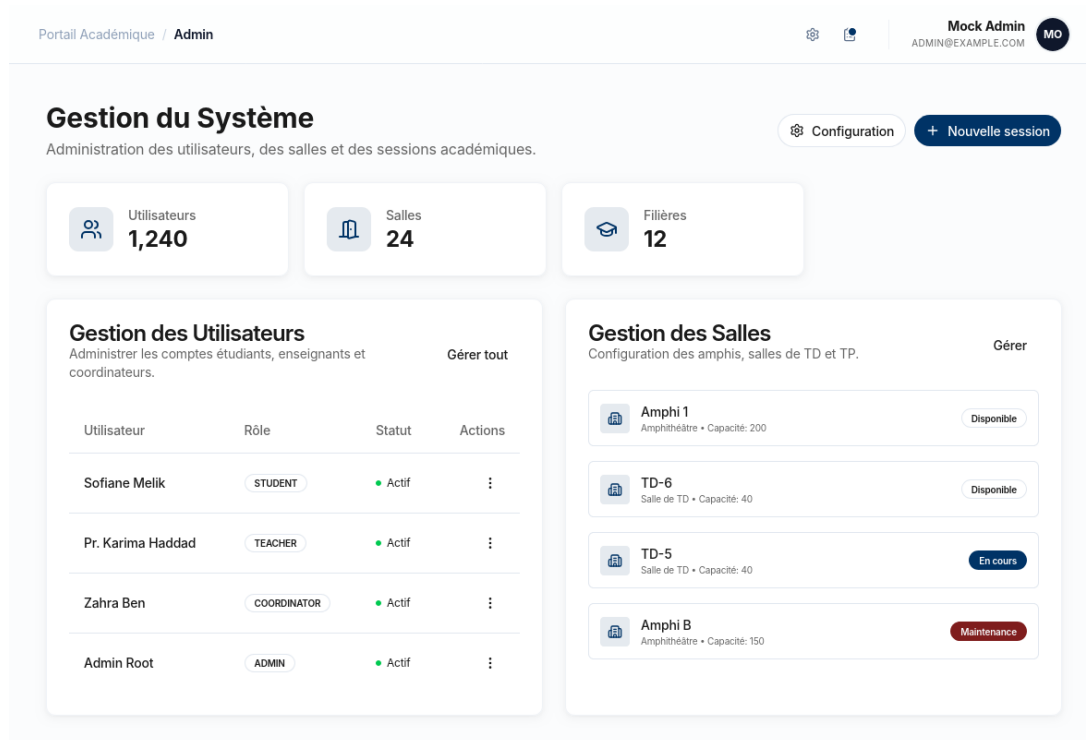


FIGURE 26 – Interface : Tableau de bord Administrateur

Tableau de bord Coordinateur. Centre névralgique de la planification, cette vue permet au coordinateur d'importer les données, d'orchestrer les sessions et de visualiser l'état d'avancement.

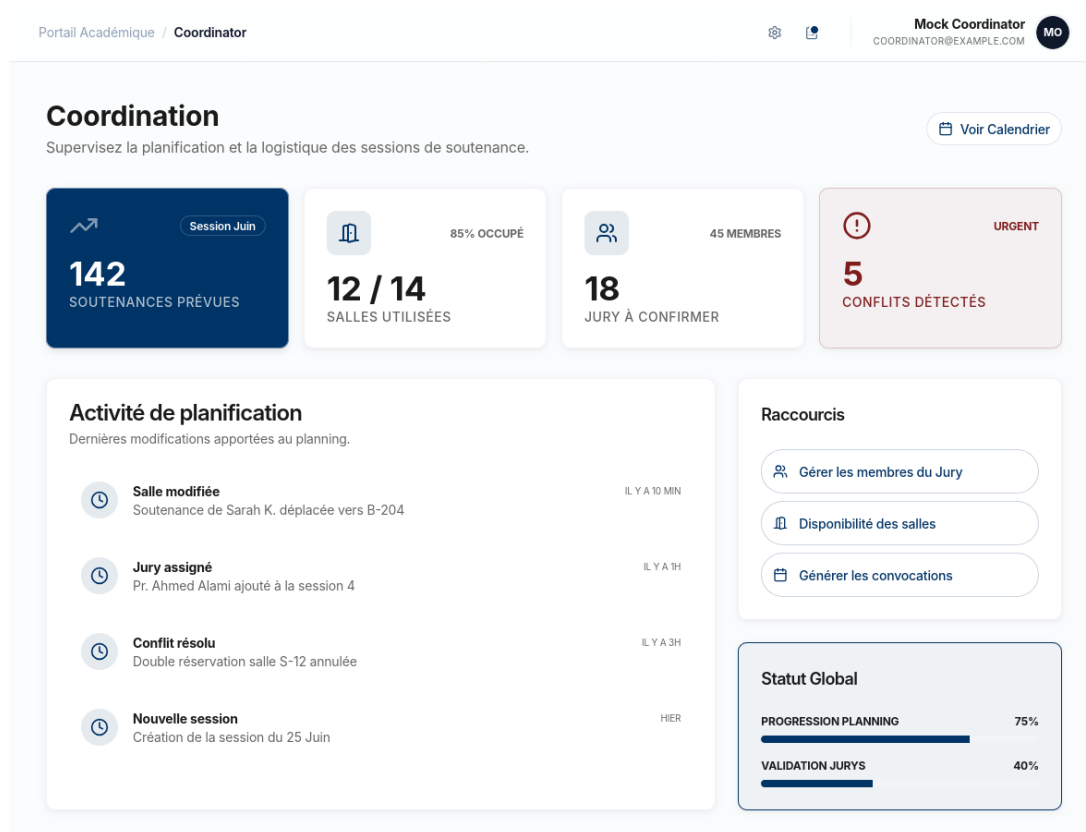


FIGURE 27 – Interface : Tableau de bord Coordinateur

Espace Enseignant. Cette interface permet aux enseignants de consulter leur planning de jury, de télécharger leurs convocations et d'accéder aux formulaires de notation.

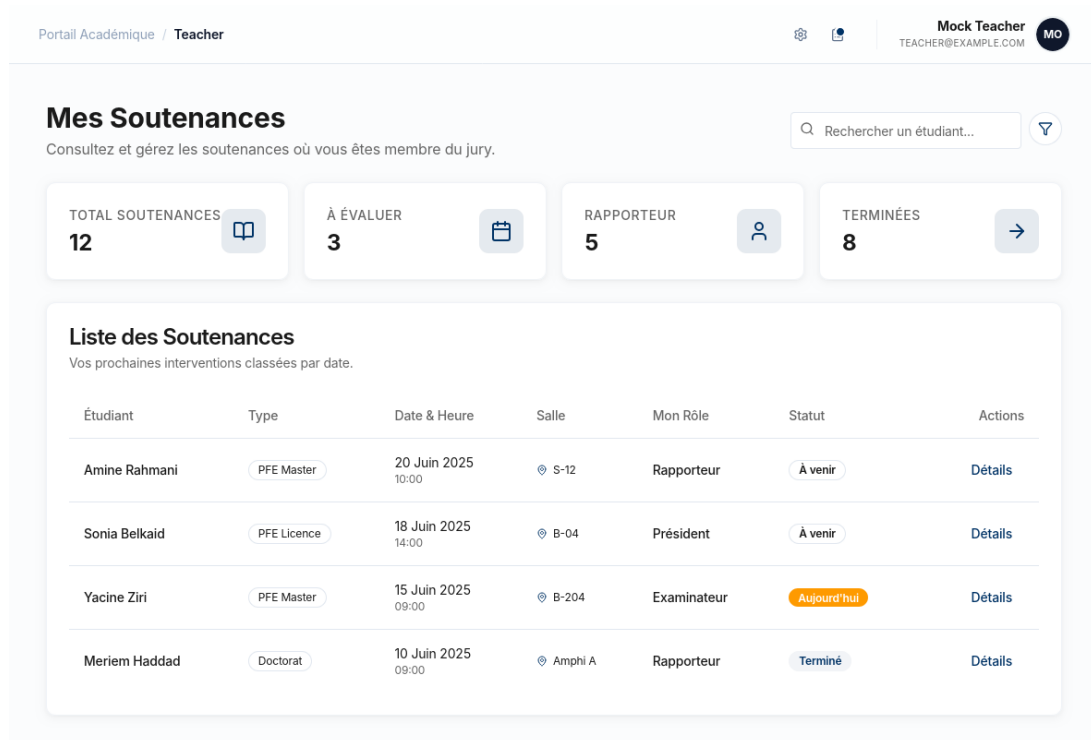


FIGURE 28 – Interface : Tableau de bord Enseignant

Espace Étudiant. L'étudiant dispose d'une interface épurée affichant les informations cruciales : date, heure, salle et composition du jury, avec téléchargement de la convocation.

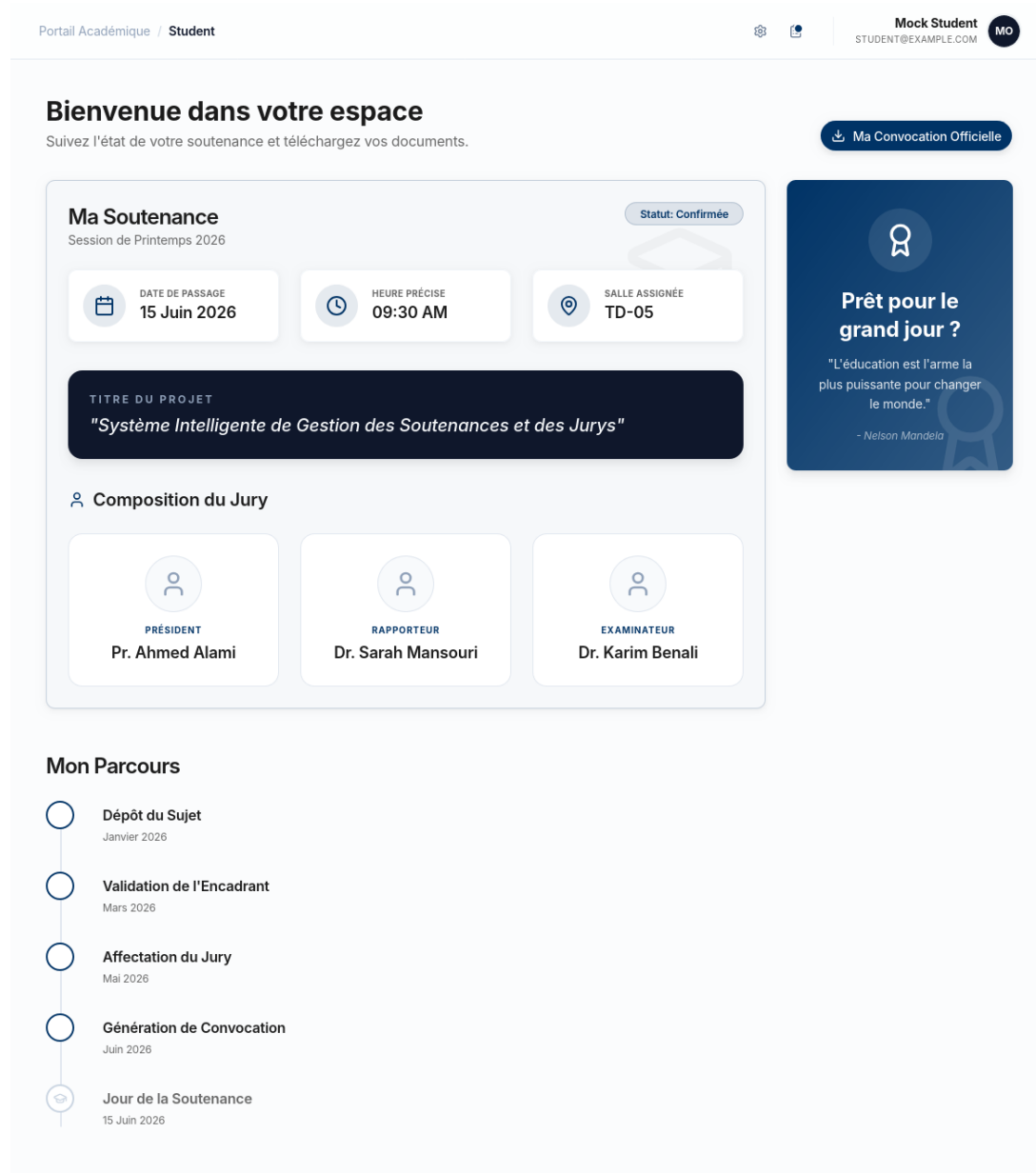


FIGURE 29 – Interface : Tableau de bord Étudiant

3.9.3 Modèle de convocation officielle

La convocation est le document administratif pivot du système. Elle constitue l'acte officiel qui déclenche la phase d'exécution des soutenances. L'automatisation complète de ce document via le service dédié apporte plusieurs garanties :

- **Intégrité des données :** les informations sont extraites directement de la base après validation des conflits, éliminant les erreurs humaines.
- **Valeur administrative :** justificatif officiel d'absence pour l'étudiant et convocation de mission pour les enseignants.
- **Réactivité :** génération instantanée dès le verrouillage de la session par le coordinateur.

CONVOCATION DE SOUTENANCE

Projet de Fin d'Études (PFE)

Il est porté à la connaissance de l'étudiant(e) **Mohamed Ali** inscrit(e) en **SMI - Sciences Mathématiques et Informatique**, que sa soutenance est programmée pour :

Date :	15 Juin 2026
Heure :	09:30 AM
Lieu :	Salle TD-05

Sujet :

"Système Intelligente de Gestion des dépenses et des budgets"

Composition du Jury :

Président	Pr. Ahmed Alami
-----------	------------------------

Rapporteur	Dr. Sarah Mansouri
------------	---------------------------

Examineur	Dr. Karim Benali
-----------	-------------------------

FIGURE 30 – Spécimen de convocation officielle (Format A4)

3.10 Conclusion

Ce chapitre a présenté l'analyse complète des besoins du système, depuis la définition des acteurs et des fonctionnalités jusqu'à la modélisation UML détaillée. Les règles de gestion qui encadrent le système ont été formalisées, les sept modules fonctionnels ont été décrits, et l'architecture technique – couches MVC, composants, sécurité JWT – a été spécifiée. Le modèle de données relationnel a été défini avec son dictionnaire complet. Enfin, la conception de l'interface utilisateur a été illustrée à travers les maquettes des écrans principaux et le modèle de convocation.

4 Réalisation et développement

4.1 Introduction

Ce chapitre présente l'implémentation technique du système. Il décrit l'environnement de développement, l'architecture détaillée du code, l'implémentation des différents modules fonctionnels ainsi que les difficultés rencontrées et les solutions apportées.

4.2 Environnement de développement

Le développement a été réalisé avec les outils et configurations suivants :

- **Système d'exploitation** : Windows 11 / Linux.
- **IDE Frontend** : VS Code avec extensions React, Tailwind CSS, ESLint.
- **IDE Backend** : IntelliJ IDEA Ultimate avec plugins Spring Boot, JPA, Maven.
- **Gestion de version** : Git et GitHub pour le versionnement du code source.
- **Test API** : Postman pour la validation des endpoints REST.

4.3 Implémentation du backend (Spring Boot)

Le backend est structuré selon l'architecture Spring Boot en couches :

- **Contrôleurs** : exposent les endpoints REST pour chaque module.
- **Services** : contiennent la logique métier et les règles de gestion.
- **Repositories** : couche d'accès aux données via Spring Data JPA.
- **Entités** : classes Java représentant les tables de la base de données.
- **Security** : configuration Spring Security avec filtres JWT.

4.4 Implémentation du frontend (React)

Le frontend est développé avec React et Tailwind CSS :

- **Composants** : architecture modulaire avec des composants réutilisables.
- **Routage** : React Router pour la navigation entre les vues.
- **État** : React Context et hooks pour la gestion d'état.
- **API** : Axios pour les appels HTTP vers le backend.
- **UI** : Shadcn UI pour les composants d'interface standardisés.

4.4.1 Architecture détaillée du backend

Le backend Spring Boot adopte une architecture en paquets métier strictement séparée par rôle fonctionnel, garantissant une faible couplage et une haute cohésion.

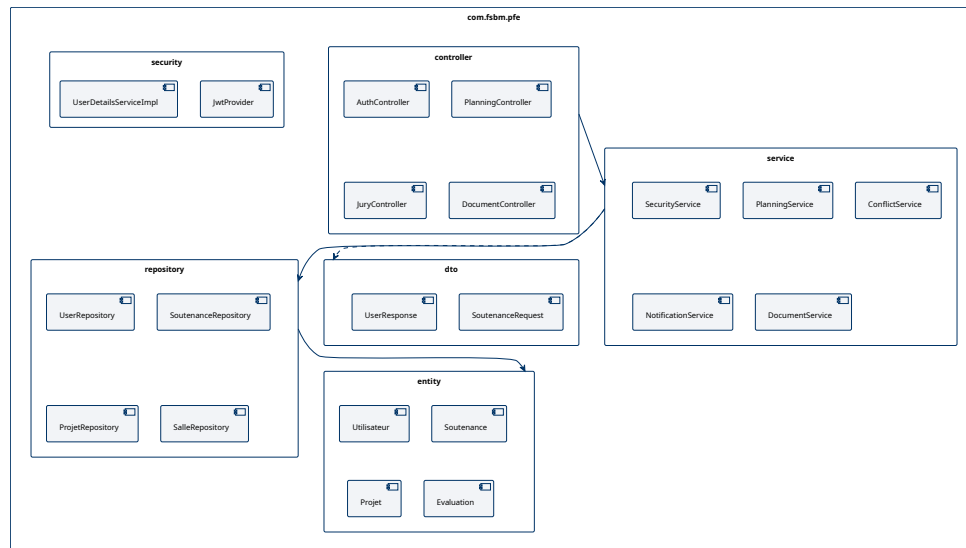


FIGURE 31 – Architecture en paquets du backend Spring Boot

Le paquetage racine 'com.system_gestion_soutenance.api' est organisé comme suit :

- **auth/** : Gestion de l'authentification (Login, JWT, mot de passe oublié)
- **user/** : Hiérarchie des utilisateurs avec héritage JOINED (**User** → Student/Teacher/Coordinator)
- **admin/** : Fonctionnalités réservées à l'administrateur
 - **session/** : Sessions globales académiques
 - **defensesession/** : Sessions de soutenance spécifiques
 - **room/** : Gestion des salles
 - **department/** : Départements pédagogiques
 - **config/** : Configuration système (niveaux, filières, grades, templates jury, emails, documents)
 - **audit/** : Journalisation des actions
 - **stats/** : Statistiques globales
- **coordinator/** : Espace coordinateur
 - **project/** : Gestion des projets et sujets
 - **group/** : Groupes d'étudiants
 - **jury/** : Composition des jurys
 - **schedule/** : Planification et créneaux
 - **conflict/** : **Détection de conflits** (8 vérifications)
 - **unavailability/** : Gestion des indisponibilités enseignants
- **teacher/** : Espace enseignant/membre de jury
 - **evaluation/** : Saisie des notes et commentaires
 - **schedule/** : Consultation du planning personnel
- **student/** : Espace étudiant
 - **defense/** : Informations de soutenance
 - **document/** : Dépôt des rapports
 - **convocation/** : Téléchargement convocation

- `notification/` : Notifications internes et emails
- `common/` : Configurations transverses
 - `config/` : `SecurityConfig`, `OpenApiConfig`

4.4.2 Architecture détaillée du frontend

Le frontend React (TypeScript) est structuré en couches logiques bien définies :

[Placeholder : Diagramme d'architecture frontend]

FIGURE 32 – Architecture du frontend React

Structure des répertoires principaux :

- `pages/` : 25+ pages organisées par rôle (Admin, Coordinator, Teacher, Student, Print)
- `components/` : Composants réutilisables
 - `ui/` : 39 composants primitifs Shadcn/Radix (button, card, dialog, data-table, side-bar...)
 - `academic/` : 5 composants métier (ProjectDialog, CreateJuryDialog, ConvocationDialog...)
 - `print/` : 8 composants dédiés à l'impression (Convocation, EvaluationSheet, AttendanceList, ProcesVerbal...)
- `hooks/` : Hooks React Query personnalisés par rôle
 - `use-admin-queries.ts`, `use-coordinator-queries.ts`, etc.
- `lib/` : Logique métier et couche API
 - `api-core.ts` : Wrapper fetch centralisé avec Bearer token
 - `conflict-engine.ts` : 8 vérifications de conflits (côté client)
 - `validations.ts` : Schémas Zod pour tous les formulaires
- `contexts/` : AuthContext (état d'authentification global)
- `mocks/` : MSW (Mock Service Worker) pour le développement offline
 - Base de données in-memory complète
 - Handlers pour tous les endpoints

4.5 Implémentation des modules fonctionnels

Cette section détaille l'implémentation technique des sept modules fonctionnels définis dans le chapitre de conception.

4.5.1 Module 1 : Authentification et gestion des rôles

L'authentification repose sur le standard **JWT (JSON Web Token)** avec une architecture stateless.

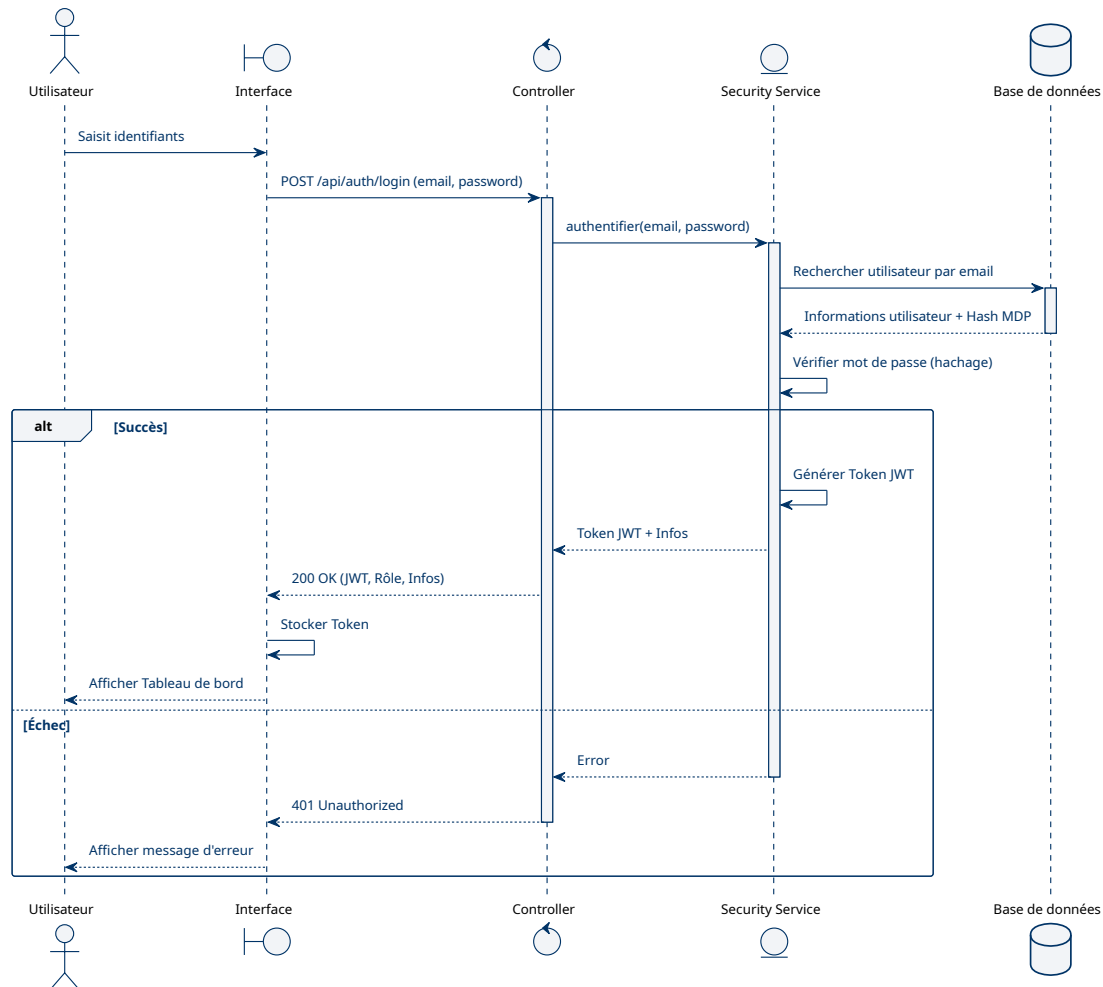


FIGURE 33 – Diagramme de séquence : Processus d'authentification JWT

Flux d'authentification :

1. L'utilisateur soumet email + password
2. Backend : BCryptPasswordEncoder valide le hash
3. JwtTokenProvider génère un token signé (HMAC-SHA256) valide 2 heures
4. Claims inclus : sub=userId, role=ADMIN|COORDINATOR|TEACHER|STUDENT
5. Le frontend stocke le token dans localStorage
6. Chaque requête ultérieure inclut l'en-tête Authorization: Bearer <token>

Sécurité côté backend (SecurityConfig.java) :

- **Stateless** : Aucune session côté serveur
- **Role-based access control** :
 - /api/admin/** → ROLE_ADMIN
 - /api/coordinator/** → ROLE_COORDINATOR
 - /api/teacher/** → ROLE_TEACHER

- `/api/student/**` → `ROLE_STUDENT`
- **Endpoints publics** : `/api/login`, `/api/auth/**`, `/swagger-ui/**`, `/h2-console/**`
- Custom handlers pour 401 Unauthorized et 403 Access Denied

4.5.2 Module 2 : Gestion académique

Ce module centralise la gestion des étudiants, enseignants, projets, groupes et configurations académiques.

- **Hiérarchie User** : Héritage JOINED avec discriminant par rôle
- **Entités de configuration** : Major (filière), Level (niveau), Grade, Department
- **Projets** : Association ManyToMany avec Student via Group
- **Import CSV** : Prévu pour utilisateurs et salles (bulk create)

4.5.3 Module 3 : Planification

Le cœur de l'application : création des créneaux, réservation des salles, génération automatique optionnelle.

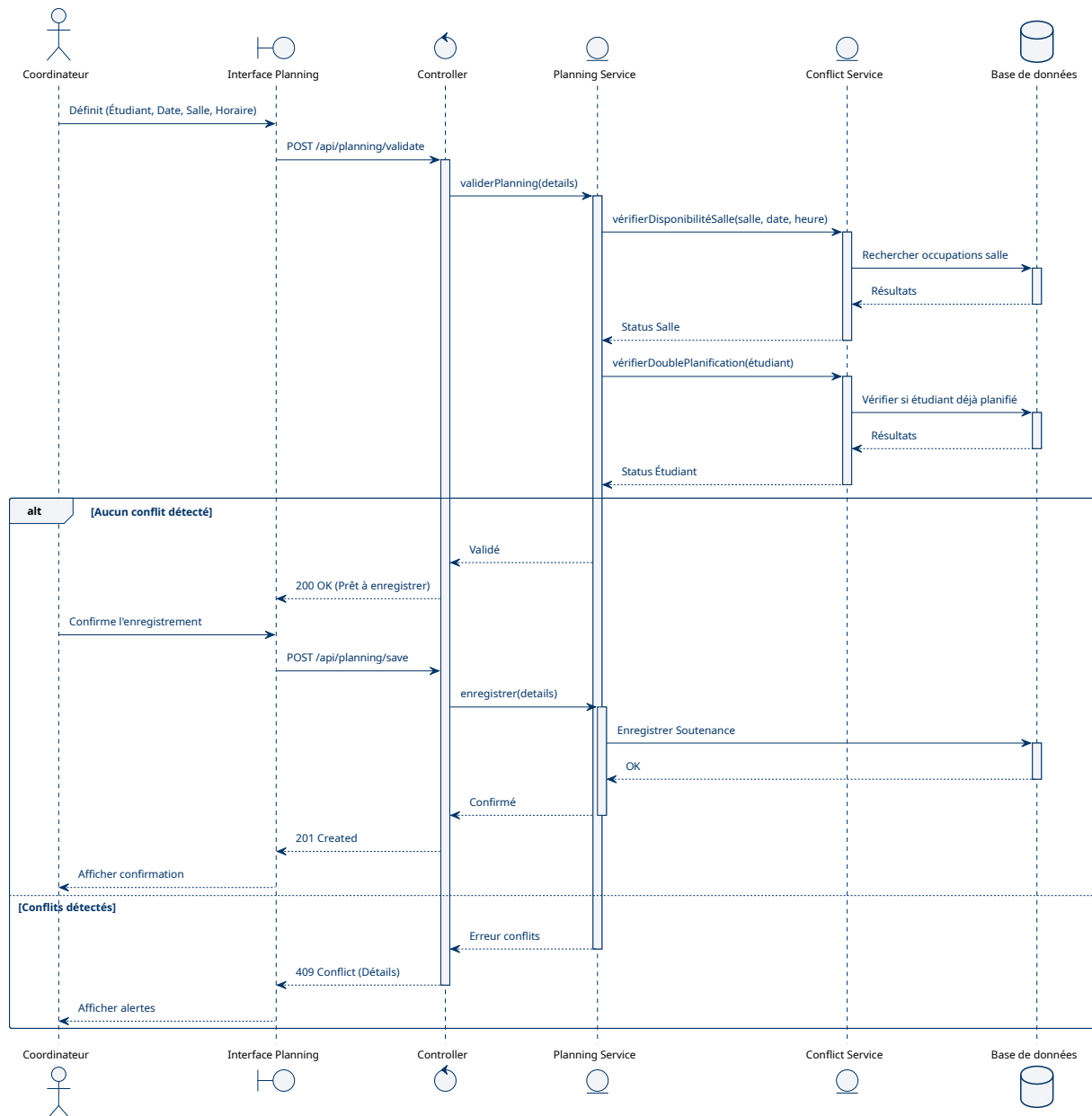


FIGURE 34 – Diagramme de séquence : Processus de planification

Entité clé : SlotAssignment

- date, time : Créneau horaire
- projectId : Projet associé
- room : Salle réservée

Fonctionnalités implémentées :

- GET/POST /api/coordinator/schedule : Consultation et modification
- POST /auto-generate : Algorithme itératif qui peuple le planning
- POST /publish : Verrouille le planning et crée des notifications

4.5.4 Module 4 : Gestion des jurys

Composition des jurys avec respect strict des Règles de Gestion RG4-RG5.

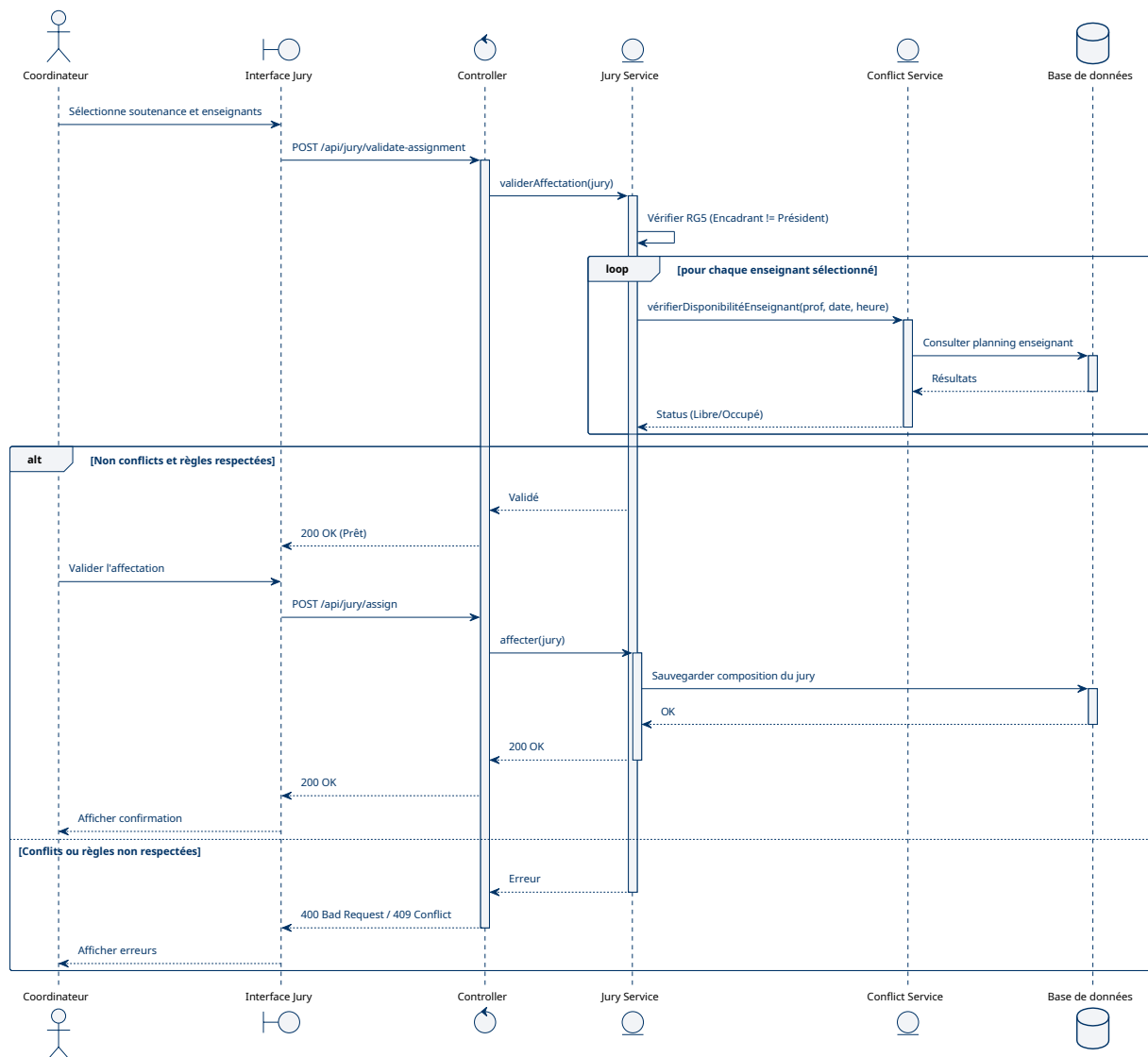


FIGURE 35 – Diagramme de séquence : Affectation d'un jury

Modèle de données :

- Jury : Entité container
- JuryMember : Ligne d'association
 - roleName : "PRESIDENT", "RAPPORTEUR", "EXAMINATEUR", "SUPERVISOR"
 - teacher : ManyToOne vers Teacher
- JuryRoleTemplate : Templates configurables par type de soutenance

Moteur de détection de conflits (8 vérifications) Le service ConflictDetectionService effectue systématiquement ces vérifications :

Conflit	Vérification
project_already_scheduled	Un projet ne peut avoir deux créneaux
slot_occupied	Même date+heure+salle déjà prise
room_capacity	Capacité de la salle respectée
date_out_of_bounds	Date dans la session globale
teacher_double_booked	Enseignant pas deux soutenances simultanées
supervisor_conflict	Encadrant disponible
break_violation	Pause entre deux soutenances
teacher_unavailable	Respect des indisponibilités déclarées

TABLE 24 – Types de conflits détectés

Ce moteur est implémenté **à la fois** backend (`ConflictDetectionService.java`) et frontend (`conflict-engine.ts`) pour une validation en temps réel lors du drag-and-drop.

4.5.5 Module 5 : Évaluation

Saisie des notes par les membres du jury.

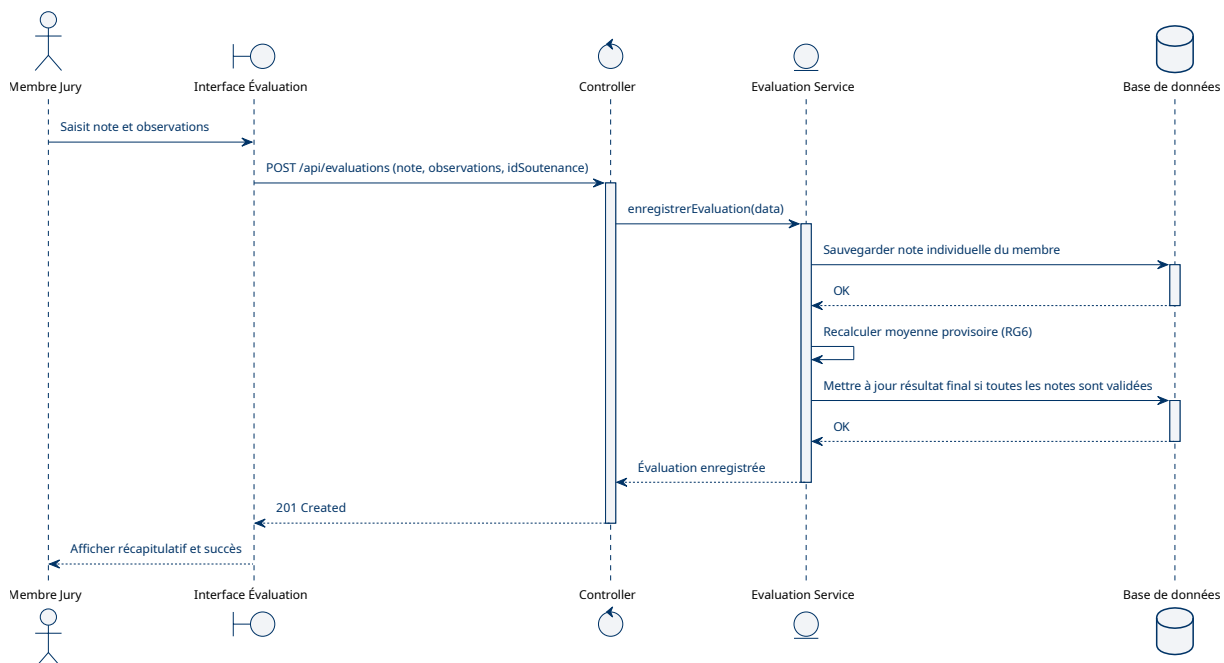


FIGURE 36 – Diagramme de séquence : Saisie d'une évaluation

Entité Evaluation :

- `teacherId` : Auteur de l'évaluation
- `projectId` : Projet concerné
- `role` : Rôle dans le jury (Président, Rapporteur...)
- `score` : Note numérique

- **comment** : Observations textuelles
- **status** : DRAFT / SUBMITTED (soumission unique)

Les coefficients de pondération par rôle sont configurables dans `DefenseSession.evaluationCoefficient`.

4.5.6 Module 6 : Génération documentaire

Deux approches complémentaires :

Backend : Convocation endpoint Endpoint GET `/api/student/convocation` :

- Actuellement retourne un placeholder texte
- Prêt pour implémentation PDF (iText ou Apache PDFBox)

Frontend : Composants Print 7 pages React dédiées sous `/print/*` :

- `/print/evaluation-sheet`
- `/print/attendance-list`
- `/print/jury-convocation`
- `/print/schedule`
- `/print/proces-verbal`
- `/print/student-convocation`
- `/print/certificate`

Ces composants utilisent des styles CSS `@media print` pour une sortie PDF de qualité via le navigateur (Ctrl+P).

4.5.7 Module 7 : Tableau de bord

Tableaux de bord différenciés par rôle :

- **Admin Dashboard** : Statistiques globales, compteurs utilisateurs, graphiques Recharts
- **Coordinator Dashboard** : Vue d'ensemble session, avancement planification
- **Teacher Dashboard** : Prochaines soutenances, évaluations en attente
- **Student Dashboard** : Informations de sa soutenance, composition jury

4.6 Extraits de code représentatifs

4.6.1 1. Génération JWT (Backend)

```

1 // JwtTokenProvider.java
2 @Service
3 public class JwtTokenProvider {
4
5     @Value("${app.jwt-secret:s3cr3t-k3y-f0r-d3f3ns3-m4n4g3m3nt-syst3m-2026}")
6     private String jwtSecret;
7
8     @Value("${app.jwt-expiration-ms:7200000}") // 2 hours
9     private int jwtExpirationMs;
10
11     public String generateToken(Integer userId, Role role) {

```

```

12     Date now = new Date();
13     Date expiryDate = new Date(now.getTime() + jwtExpirationMs);
14
15     return Jwts.builder()
16         .subject(userId.toString())
17         .claim("role", role.name())
18         .issuedAt(now)
19         .expiration(expiryDate)
20         .signWith(getSignInKey())
21         .compact();
22 }
23
24 private SecretKey getSignInKey() {
25     byte[] keyBytes = Decoders.BASE64.decode(
26         java.util.Base64.getEncoder().encodeToString(jwtSecret.getBytes())
27     );
28     return Keys.hmacShaKeyFor(keyBytes);
29 }
30 }

```

4.6.2 2. Vérification de conflit : Double réservation enseignant

```

1 // ConflictDetectionService.java - extrait
2 @Service
3 public class ConflictDetectionService {
4
5     public List<ConflictResult> validateSlotAssignment(SlotAssignment slot) {
6         List<ConflictResult> conflicts = new ArrayList<>();
7
8         // Récupère le projet et son jury
9         Optional<Project> project =
10             ↪ projectRepository.findById(slot.getProjectId());
11         if (project.isEmpty()) return conflicts;
12
13         // Vérifie chaque membre du jury
14         List<JuryMember> members =
15             ↪ juryMemberRepository.findByJury_ProjectId(slot.getProjectId());
16
17         for (JuryMember member : members) {
18             LocalTime slotStart = slot.getTime();
19             LocalTime slotEnd = slotStart.plusMinutes(DEFAULT_DURATION);
20
21             // Cherche d'autres créneaux pour ce même enseignant à la même
22             ↪ date
23             List<SlotAssignment> teacherSlots = slotAssignmentRepository
24                 .findByDateAndJuryMembersContaining(slot.getDate(),
25                 ↪ member.getTeacher());
26
27             for (SlotAssignment existing : teacherSlots) {
28                 if (existing.getId().equals(slot.getId())) continue;
29             }
30         }
31     }
32 }

```

```

26     LocalTime existingStart = existing.getTime();
27     LocalTime existingEnd =
    ↪     existingStart.plusMinutes(DEFAULT_DURATION);
28
29     // Vérifie chevauchement
30     if (timesOverlap(slotStart, slotEnd, existingStart,
    ↪     existingEnd)) {
31         conflicts.add(ConflictResult.builder()
32             .type("teacher_double_booked")
33             .severity(Severity.ERROR)
34             .teacher(member.getTeacher())
35             .conflictingSlot(existing)
36             .message("Enseignant " +
    ↪     member.getTeacher().getUser().getFullName() +
37                 " déjà affecté à " + existing.getTitle() +
38                 " (" + existing.getTime() + ")")
39             .build());
40     }
41 }
42 }
43 return conflicts;
44 }
45 }

```

4.6.3 3. React Query : Hook personnalisé (Frontend)

```

1 // use-coordinator-queries.ts - extrait
2 import { useQuery, useMutation, useQueryClient } from
    ↪ '@tanstack/react-query';
3 import {
4     getProjects, createProject, updateProject, deleteProject,
5     getSchedule, saveSchedule, validateSchedule, generateSchedule,
6     type CreateProjectRequest, type SlotAssignment
7 } from '@lib/api-coordinator';
8
9 export function useProjects() {
10     return useQuery({
11         queryKey: ['coordinator', 'projects'],
12         queryFn: getProjects,
13         staleTime: 30_000 // 30s
14     });
15 }
16
17 export function useCreateProject() {
18     const queryClient = useQueryClient();
19
20     return useMutation({
21         mutationFn: (data: CreateProjectRequest) => createProject(data),
22         onSuccess: () => {
23             queryClient.invalidateQueries({ queryKey: ['coordinator', 'projects']
    ↪     });

```

```
24     queryClient.invalidateQueries({ queryKey: ['coordinator', 'stats'] });
25   }
26 });
27 }
28
29 export function useSchedule() {
30   return useQuery({
31     queryKey: ['coordinator', 'schedule'],
32     queryFn: getSchedule
33   });
34 }
35
36 export function useGenerateSchedule() {
37   const queryClient = useQueryClient();
38
39   return useMutation({
40     mutationFn: () => generateSchedule(),
41     onSuccess: () => {
42       queryClient.invalidateQueries({ queryKey: ['coordinator', 'schedule']
43         ↪   });
44     }
45   });
46 }
```

4.7 Interfaces utilisateur

Cette section présente les principales vues de l'application.

Rôle	Pages principales	URL
Admin	Tableau de bord	/admin
	Gestion utilisateurs	/admin/users/*
	Salles	/admin/rooms
	Sessions académiques	/admin/sessions
	Configuration	/admin/config
	Journal d'audit	/admin/audit-logs
Coordinator	Tableau de bord	/coordinator
	Projets & Groupes	/coordinator/projects
	Defense Designer	/coordinator/schedule
	Jurys	/coordinator/juries
	Conflits	/coordinator/conflicts
	Notes	/coordinator/grades
Teacher	Tableau de bord	/teacher
	Planning	/teacher/schedule
	Évaluations	/teacher/evaluations
	Indisponibilités	/teacher/unavailability
Student	Tableau de bord	/student
	Groupe	/student/group
	Documents	/student/documents

TABLE 25 – Architecture des pages par rôle

4.7.1 Écran de connexion

Point d'entrée unique pour tous les utilisateurs avec validation des identifiants et redirection selon le rôle.

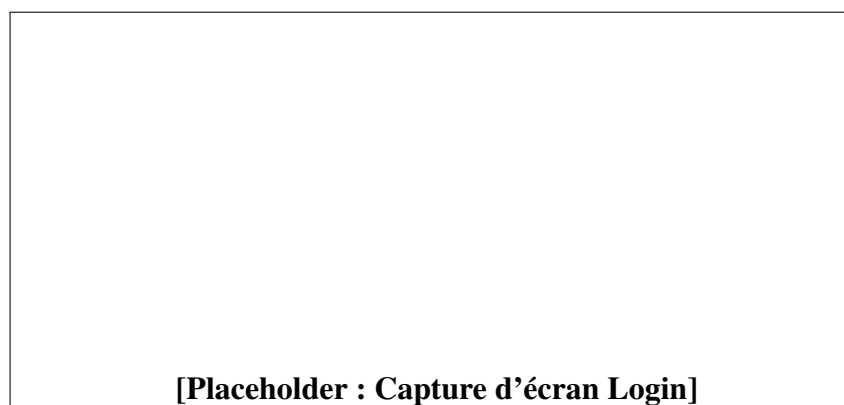


FIGURE 37 – Interface : Page de connexion

4.7.2 Defense Designer (Coordinateur)

Vue centrale de planification avec système de glisser-déposer (DnD Kit) et validation de conflits en temps réel.

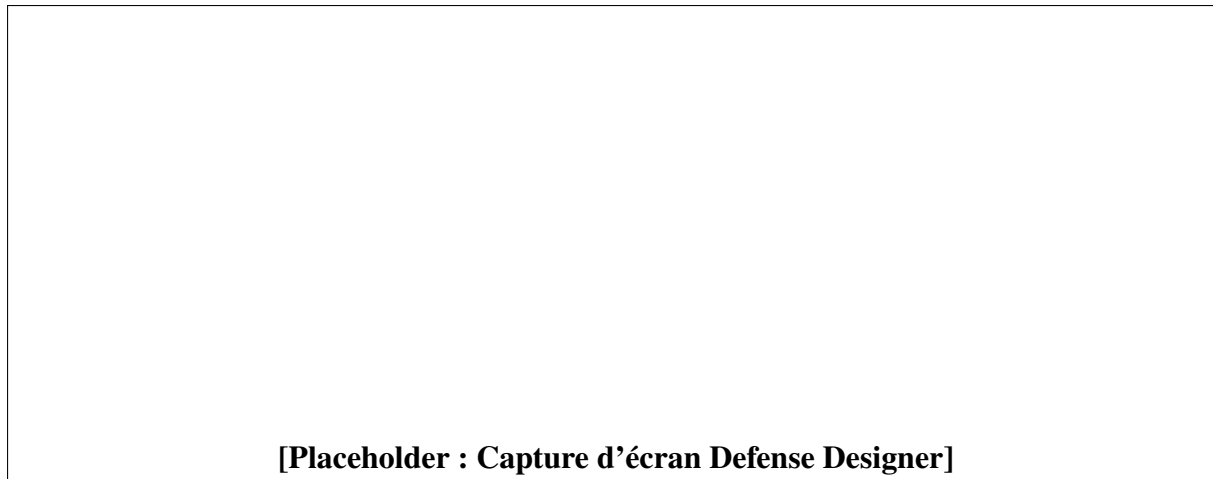


FIGURE 38 – Interface : Defense Designer (Planification)

Fonctionnalités clés de cette vue :

- Calendrier par jour / par salle
- Projets non planifiés dans la barre latérale
- Drag & drop pour assigner un projet à un créneau
- Indicateurs visuels de conflits (rouge = bloquant, orange = avertissement)
- Boutons : **Auto-generate**, **Validate**, **Publish**

4.7.3 Tableau de bord Administrateur

Vue globale avec compteurs et graphiques Recharts.

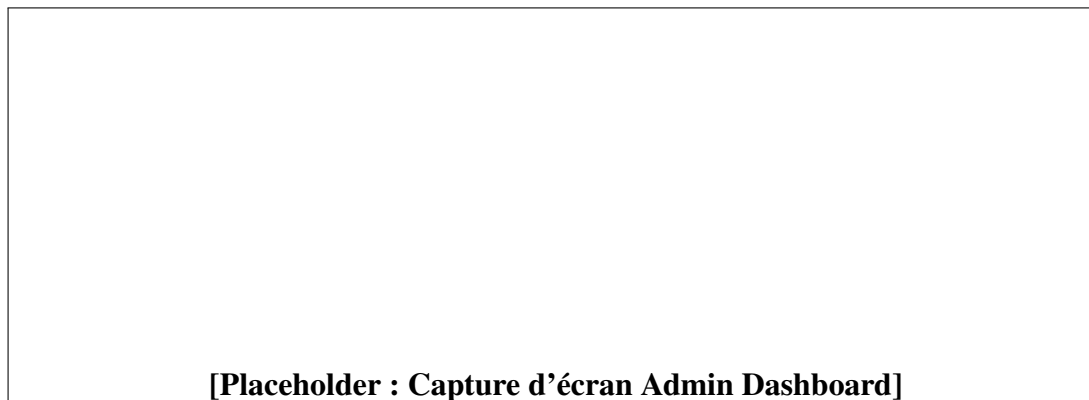


FIGURE 39 – Interface : Tableau de bord Admin

4.7.4 Formulaire d'évaluation (Enseignant)

Interface de notation par membre de jury avec champ commentaire et coefficients.

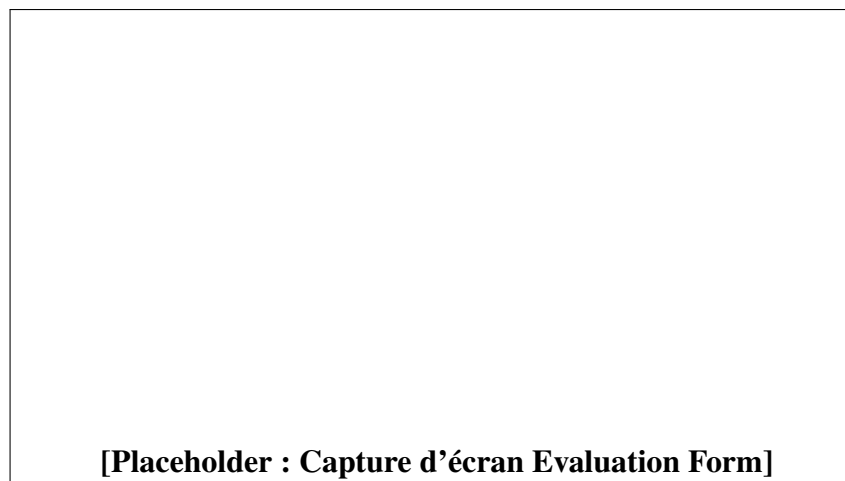


FIGURE 40 – Interface : Formulaire d'évaluation

4.7.5 Espace Étudiant

Vue simplifiée affichant les informations de soutenance : date, heure, salle, composition du jury.

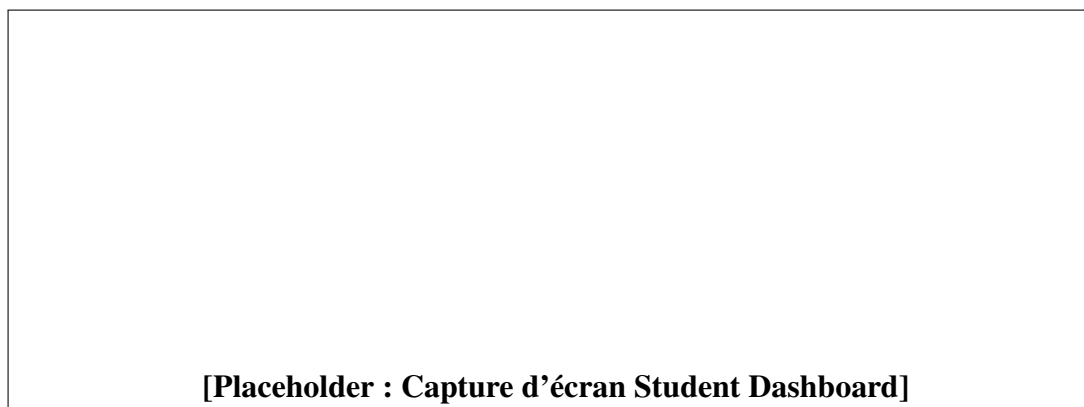


FIGURE 41 – Interface : Espace étudiant

4.7.6 Impression : Convocation

Composant `/print/student-convocation` optimisé pour export PDF (Ctrl+P).

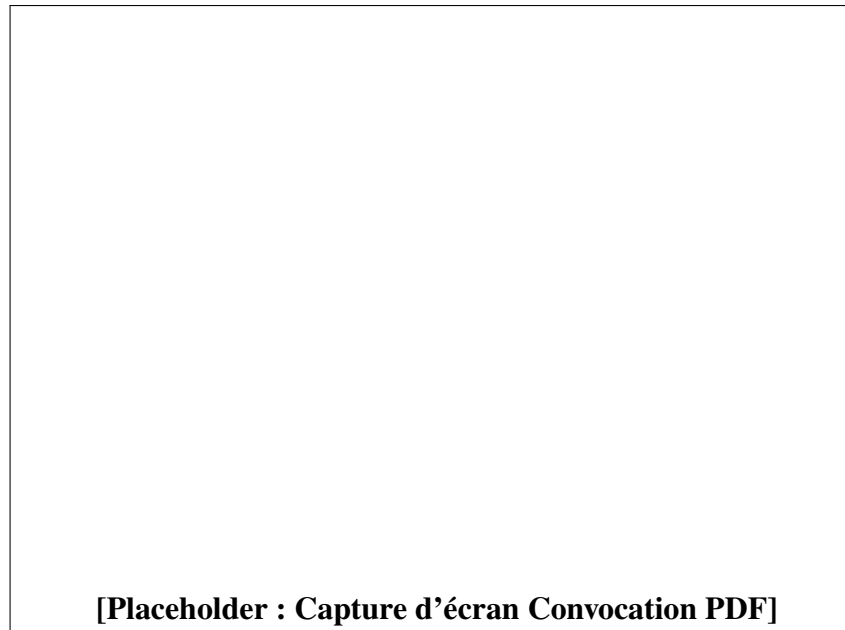


FIGURE 42 – Interface : Modèle de convocation (impression)

Autres vues d'impression disponibles :

- Feuille d'évaluation
- Liste de présence
- Planning global
- Procès-verbal
- Convocation jury

4.8 Difficultés rencontrées et solutions

Les principales difficultés rencontrées lors du développement incluent la gestion optimisée des conflits horaires et l'équilibrage automatique de la charge des jurys, résolues par des algorithmes de vérification systématique et des heuristiques de répartition.

4.9 Conclusion

Ce chapitre a présenté l'implémentation du système selon l'architecture définie au chapitre précédent. Le backend Spring Boot et le frontend React ont été structurés pour assurer la maintenabilité et l'évolutivité de l'application.

5 Tests et validation

5.1 Introduction

Ce chapitre présente la stratégie de test adoptée pour garantir la qualité et la fiabilité du système développé. Il détaille les tests fonctionnels et d'intégration, les scénarios de validation, ainsi que les critères de réussite du projet.

5.2 Stratégie de test

La stratégie de test adoptée prévoit trois niveaux complémentaires, avec une infrastructure technique déjà configurée :

- **Tests unitaires et d'intégration (Backend)** : Infrastructure **JUnit 5** et **Spring Boot Test** configurée dans le projet Maven.
- **Tests d'intégration API** : Prévus via **Postman** avec collections d'endpoints.
- **Tests End-to-End (Frontend)** : Projet séparé system-gestion-soutenance-e2e avec **Playwright** configuré.
- **Mocks développement** : **MSW (Mock Service Worker)** implémenté côté frontend pour prototypage offline.

Niveau de test	Framework	État
Tests unitaires Backend	JUnit 5 + Spring Boot Test	Infrastructure OK
Tests API	Postman	Collections à créer
Tests E2E	Playwright	Projet prêt
Tests frontend offline	MSW	Implémenté (développement)

TABLE 26 – Infrastructure de test

5.3 Scénarios de test

Les scénarios de test planifiés couvrent les fonctionnalités critiques du système. **À la date de rédaction de ce rapport, les tests automatisés ne sont pas encore implémentés** (seul un test placeholder `contextLoads()` existe dans le backend).

Tests manuels effectués: Toutes les fonctionnalités principales ont été validées **manuellement** pendant le développement :

- Connexion/déconnexion avec les 4 rôles
- Gestion CRUD complète (utilisateurs, salles, projets, jurys, créneaux)
- Planification avec détection de conflits en temps réel
- Saisie et consultation des évaluations
- Composants d'impression PDF via navigateur

Scénarios prévus pour l'automatisation future :

- **Authentification** : connexion avec identifiants valides/invalides, restriction d'accès par rôle, expiration de session.
- **Gestion des utilisateurs** : création, modification et suppression de comptes pour chaque rôle.
- **Planification** : création d'une session globale, planification d'une soutenance individuelle, modification des créneaux.
- **Affectation des jurys** : composition d'un jury avec respect des règles RG4 et RG5, équilibrage de la charge.
- **Détection de conflits** : double affectation d'un enseignant, réservation simultanée d'une salle, double planification d'un étudiant.
- **Génération de documents** : production de convocations PDF, fiches d'évaluation et planning général.
- **Saisie des évaluations** : enregistrement des notes, calcul de la moyenne (RG6), validation du résultat final.

5.4 Validation des fonctionnalités

La validation manuelle a été effectuée selon les critères suivants :

- Les utilisateurs peuvent être gérés de manière sécurisée selon leur rôle.
- Les étudiants, enseignants et salles peuvent être enregistrés et modifiés sans erreur.
- Une soutenance peut être planifiée en respectant les contraintes temporelles et les règles de gestion.
- Un jury peut être affecté avec détection automatique des conflits.
- Les composants d'impression (convocations, évaluations) sont fonctionnels et exportables en PDF via le navigateur.
- Les évaluations et notes peuvent être saisies et enregistrées de manière persistante.
- Le planning global est consultable, filtrable et exploitable par tous les acteurs.

5.5 Évaluation globale du système

L'évaluation globale du système repose actuellement sur les tests manuels effectués par les développeurs tout au long du cycle de développement.

Le **groupe pilote** – coordinateur, enseignants et étudiants représentant les rôles décrits en Section 3.3.1 – n'a pas encore été mobilisé. Cette étape de validation utilisateur est prévue pour les prochaines itérations du projet, afin de recueillir des retours sur l'ergonomie, la pertinence des fonctionnalités et la performance globale en conditions réelles.

5.6 Limites du projet

Le projet présente certaines limites qui pourront être adressées dans des versions futures :

- La planification optimisée des créneaux n'est pas entièrement automatisée ; elle repose sur les choix du coordinateur assisté par la détection de conflits.
- L'envoi des notifications par e-mail n'est pas implémenté dans la version initiale (mock seulement).

- L'import massif de données depuis Excel est limité à un format prédéfini.
- **Tests automatisés** : L'infrastructure est configurée (JUnit 5, Spring Boot Test, Playwright) mais les tests ne sont pas implémentés (seulement un test placeholder).
- **Génération PDF backend** : L'endpoint `/api/student/convocation` retourne actuellement un placeholder texte ; la génération PDF native via iText/PDFBox est à implémenter.

5.7 Conclusion

La stratégie de test définie dans ce chapitre pose les bases pour garantir la conformité du système aux exigences du cahier des charges. L'infrastructure technique – **JUnit 5, Spring Boot Test, MSW, Playwright** – est en place et prête à accueillir les tests automatisés dans les prochaines itérations.

En attendant, une validation manuelle approfondie a été effectuée sur l'ensemble des fonctionnalités principales : authentification multi-rôles, planification avec détection de conflits, gestion des jurys, saisie des évaluations et composants d'impression.

Conclusion Générale et Perspectives

Bilan du travail réalisé

Ce projet de fin d'études nous a permis de concevoir et de développer une application web complète pour la gestion des soutenances et des jurys au sein de la Faculté des Sciences Ben M'Sick. Partant d'un constat partagé – l'inefficacité des méthodes manuelles actuelles – nous avons mené une analyse approfondie de l'existant, étudié les solutions disponibles sur le marché et défini un cahier des charges précis.

La conception du système a été réalisée en suivant une démarche rigoureuse de modélisation UML, couvrant les aspects fonctionnels, statiques et dynamiques. L'architecture technique retenue – Spring Boot pour le backend, React pour le frontend, MySQL pour la persistance – offre un socle robuste, évolutif et sécurisé.

Le système couvre l'ensemble du cycle de vie d'une soutenance, de la planification des créneaux et de l'affectation des jurys jusqu'à la saisie des évaluations et la génération automatique des documents administratifs, en passant par la détection des conflits.

Apports personnels et compétences acquises

La réalisation de ce projet nous a permis de mettre en pratique les connaissances acquises durant notre formation en licence MIP, notamment en développement web, en bases de données et en génie logiciel. Nous avons approfondi notre maîtrise de technologies modernes telles que Spring Boot, React, JWT et MySQL.

Sur le plan méthodologique, ce projet nous a initiés à la gestion d'un projet informatique complet, depuis l'analyse des besoins jusqu'à la conception, en passant par la modélisation UML et la gestion de version avec Git.

Perspectives d'évolution

Plusieurs évolutions sont envisagées pour enrichir le système à moyen terme :

- **Notifications enrichies** : modèles d'e-mails avancés, rappels automatiques et suivi détaillé des envois.
- **Import avancé** : chargement massif de données académiques depuis des fichiers Excel.
- **Planification optimisée** : génération automatique des plannings fondée sur des algorithmes de satisfaction de contraintes.
- **Visioconférence intégrée** : module dédié aux soutenances à distance.
- **Signature électronique** : des procès-verbaux et archivage numérique des soutenances passées.
- **Statistiques annuelles** : par filière, avec tableaux de bord enrichis.
- **Interopérabilité** : intégration avec le système d'information global de l'établissement.

Bibliographie

Références

- [1] Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [2] React Documentation, <https://react.dev/>
- [3] JSON Web Token (JWT) RFC 7519, <https://datatracker.ietf.org/doc/html/rfc7519>
- [4] OMG Unified Modeling Language (UML) Specification, <https://www.omg.org/spec/UML/>
- [5] MySQL Documentation, <https://dev.mysql.com/doc/>
- [6] Shadcn UI Documentation, <https://ui.shadcn.com/>
- [7] Tailwind CSS Documentation, <https://tailwindcss.com/docs>
- [8] Pro Git Book, <https://git-scm.com/book/>